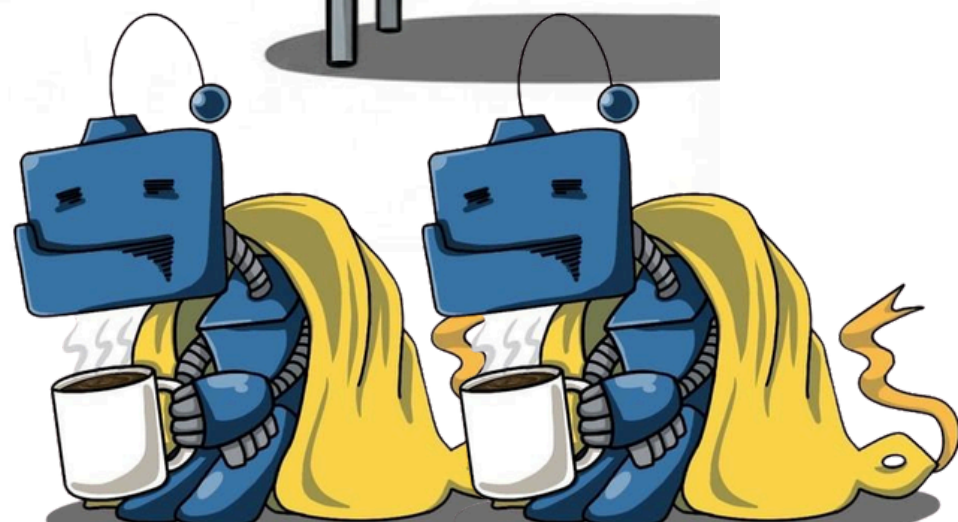
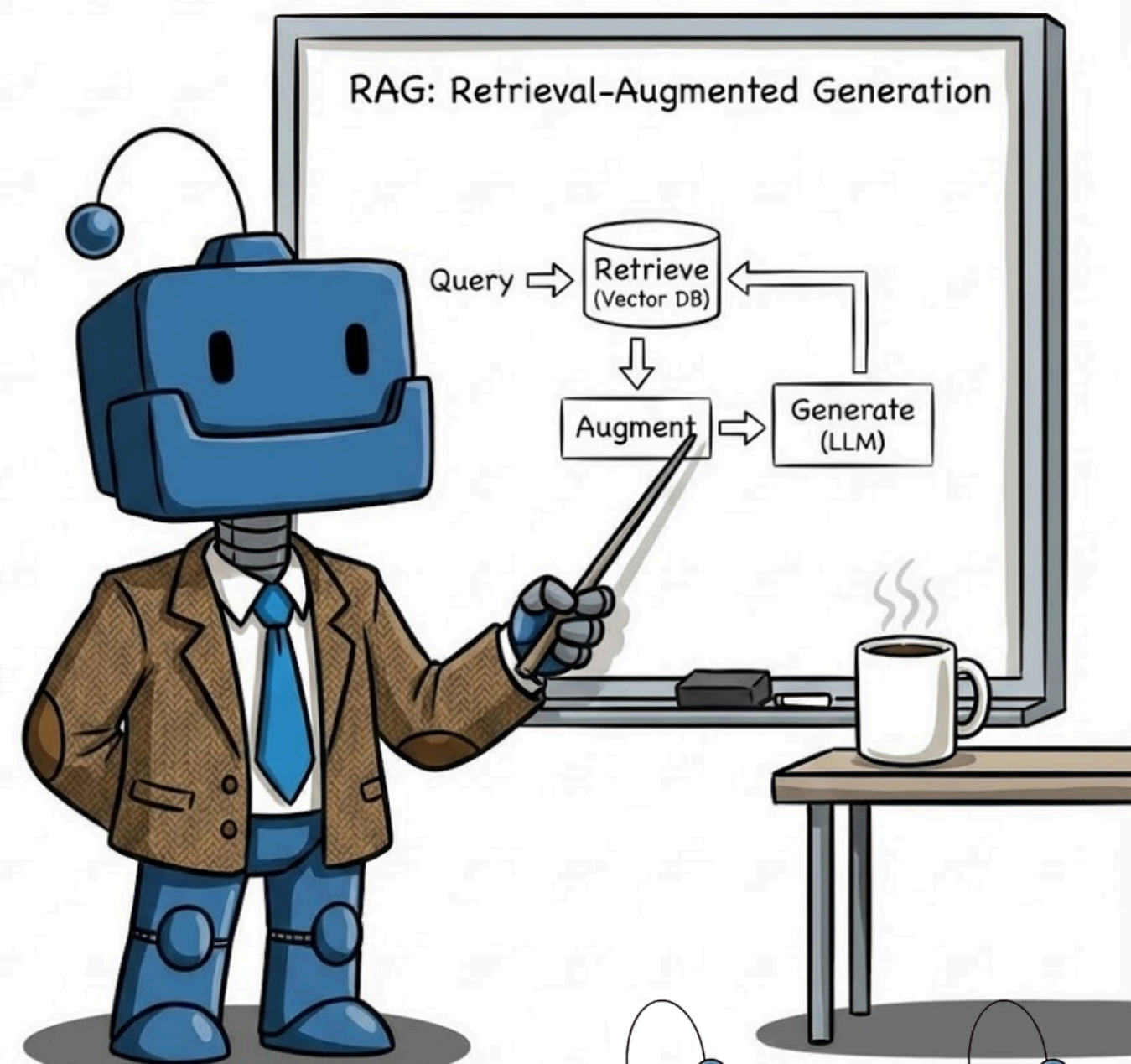
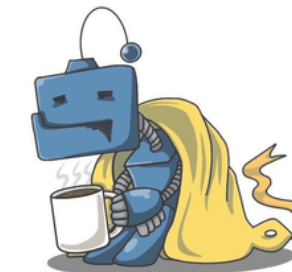
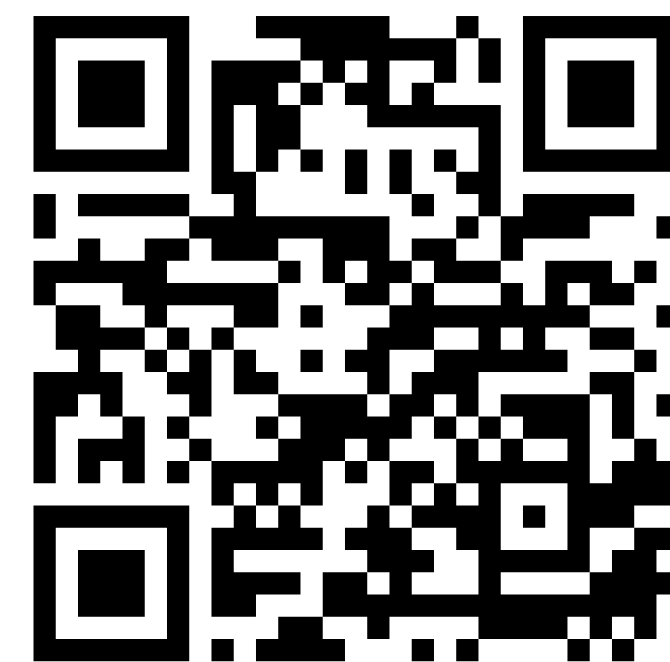




RAG BACK TO BASIC IN 2026



Natdhanai Praneenatthavee, AI engineer
Founder ทีมมัลคัด python

Concept of AI
is Not 100% Correct

nations



About Me

Natdhanai Praneenatthavee (JOB)

- Head of AI Engineer & Founder AI Startups (Experience 7 years+)
- AI Technology Advisor in Enterprise
- AI Alliance Ambassador (Meta X IBM X HF)
- Founder of เพจ ตื่นมาโค้ด python (16K followers)
- Speaker: Google, AWS, TensorFlow,
- Guest Lecturer: International School of Engineering Chulalongkorn University, College of Arts, Media and Technology, Chiang Mai University,
- Nvidia Agentic-AI Professional Certified (Official)



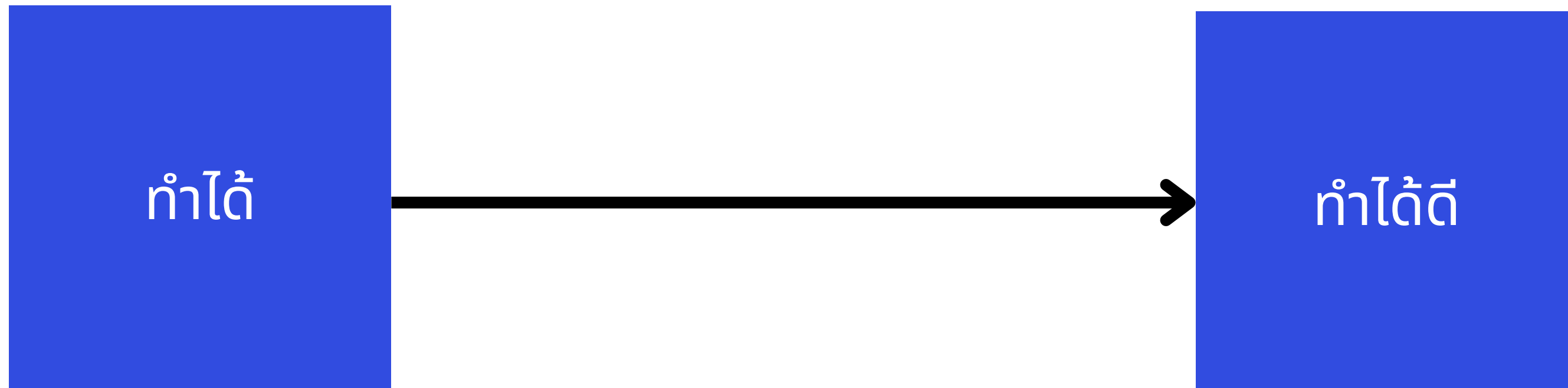
AI Journey

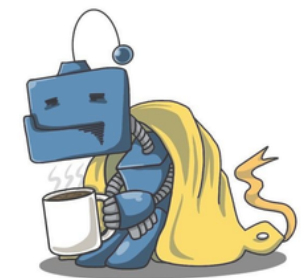


Passion

Expertise

AI Journey

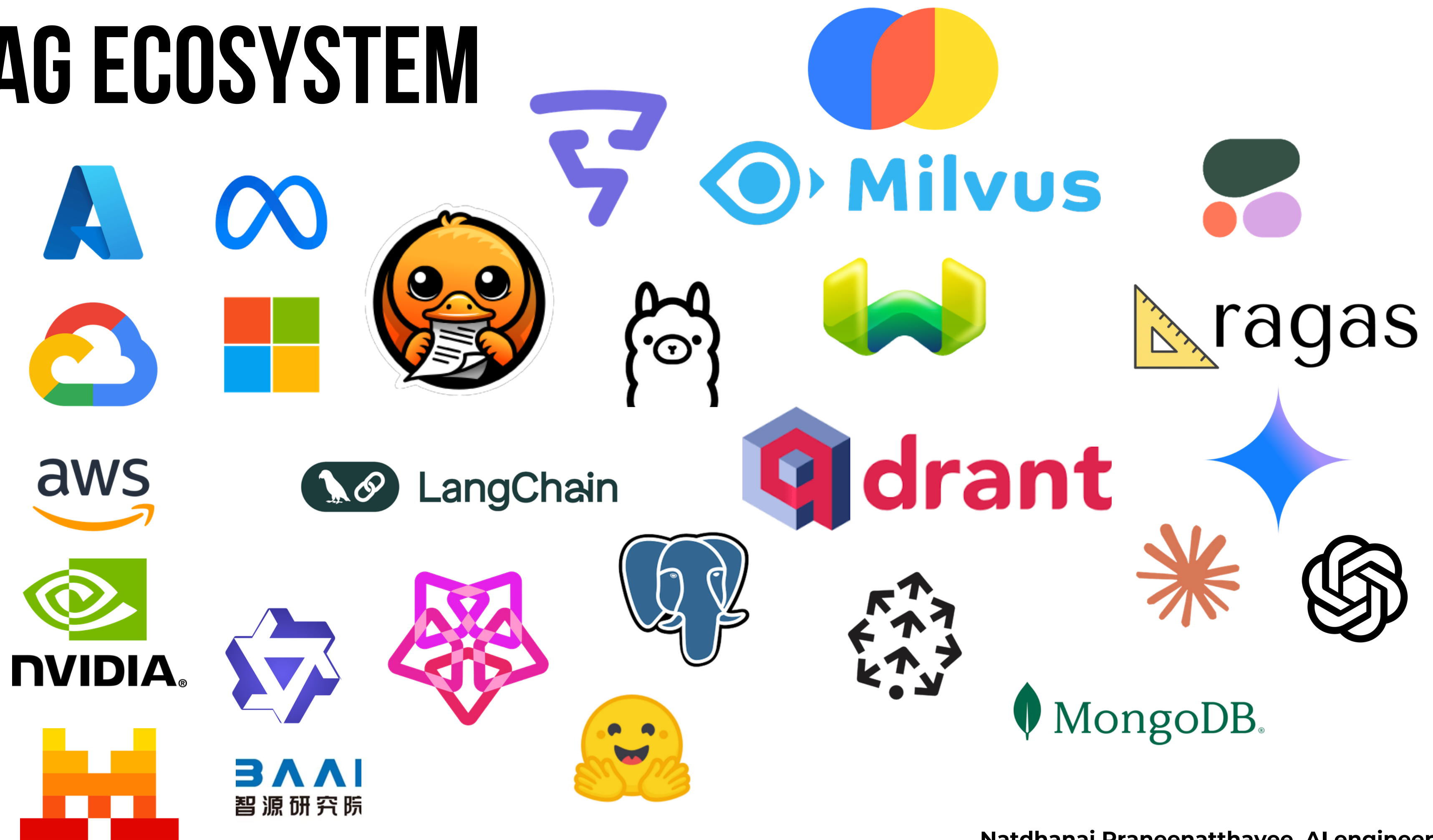




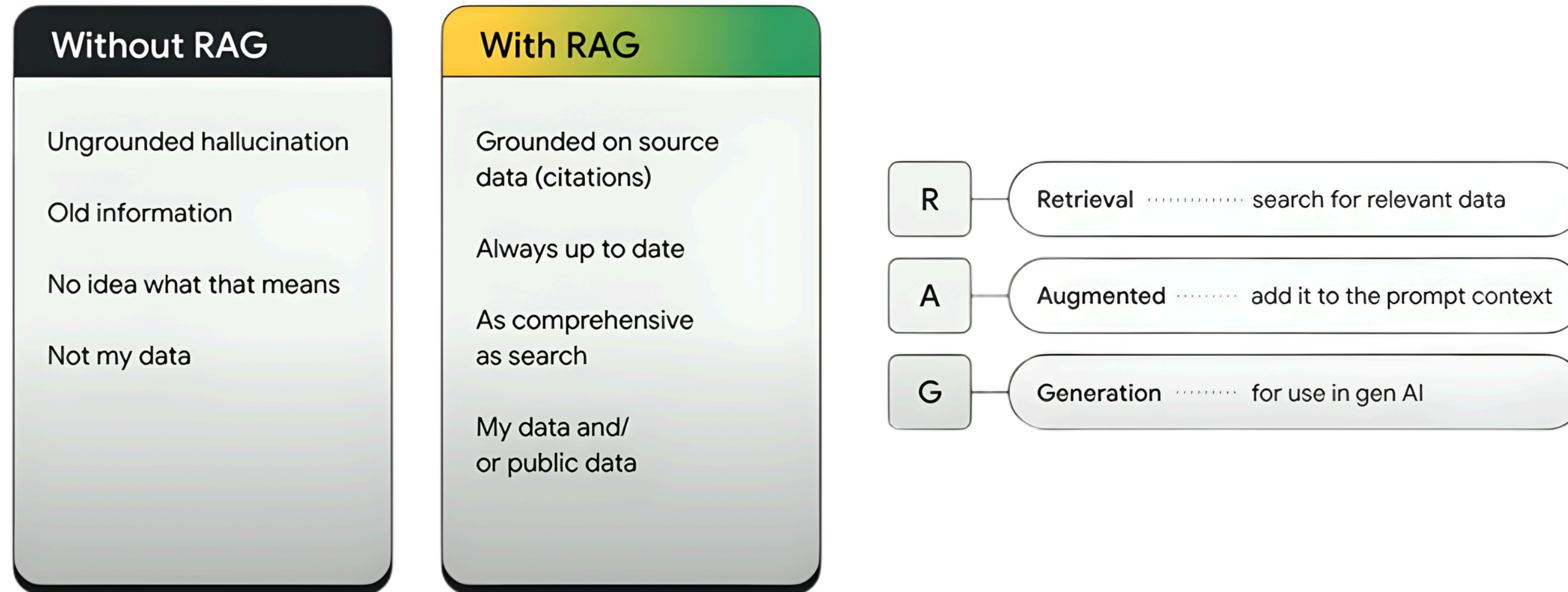
SESSION 1: UNDERSTANDING THE BASIC

Natdhanai Praneenatthavee, AI engineer
Founder ทีมมาโค้ด python

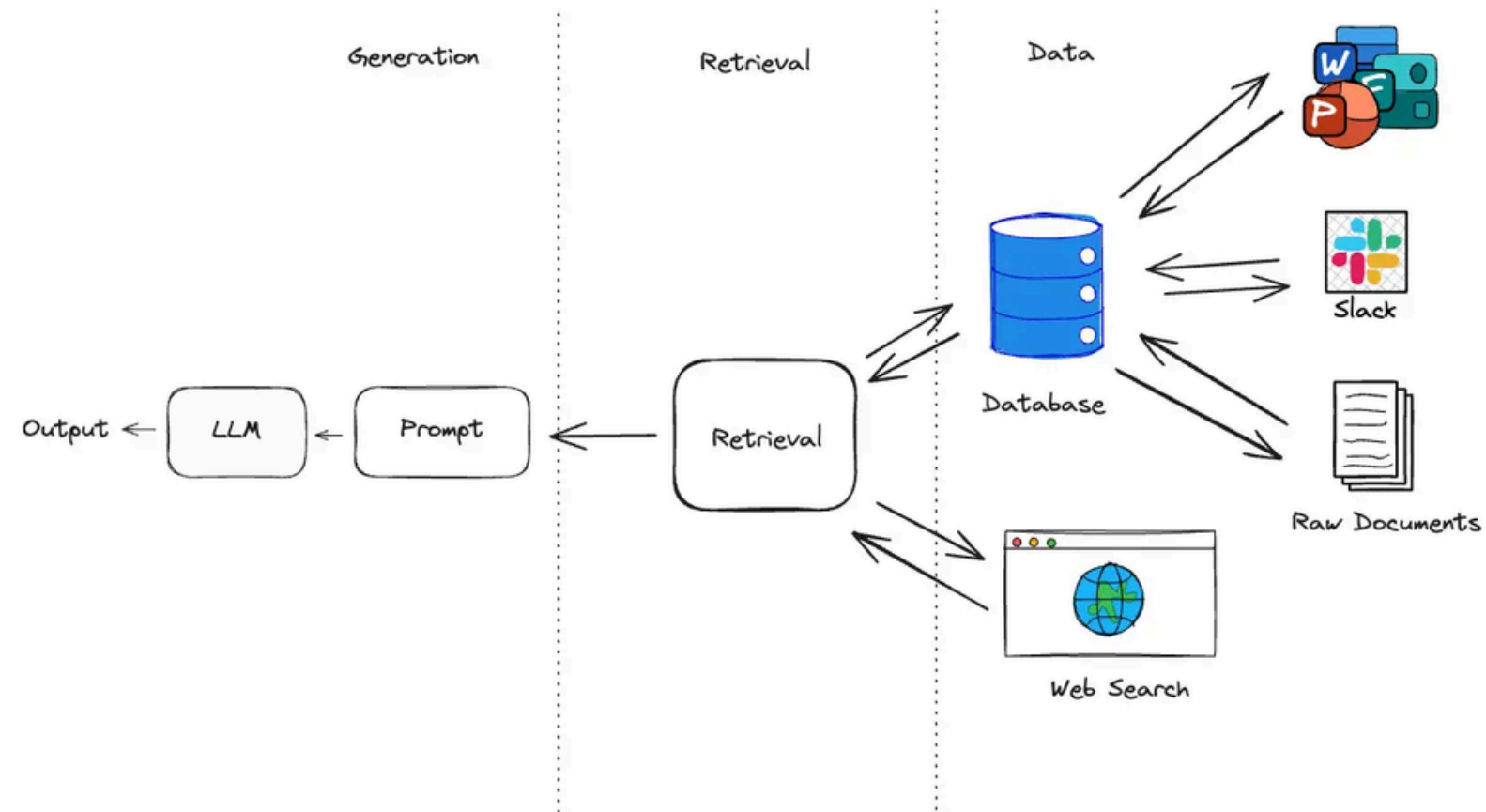
RAG ECOSYSTEM



RAG



WHAT IS RAG?



RETRIEVAL-AUGMENTED GENERATION

Retrieval

Retrieve relevant knowledge or information from the data source.

Augmented

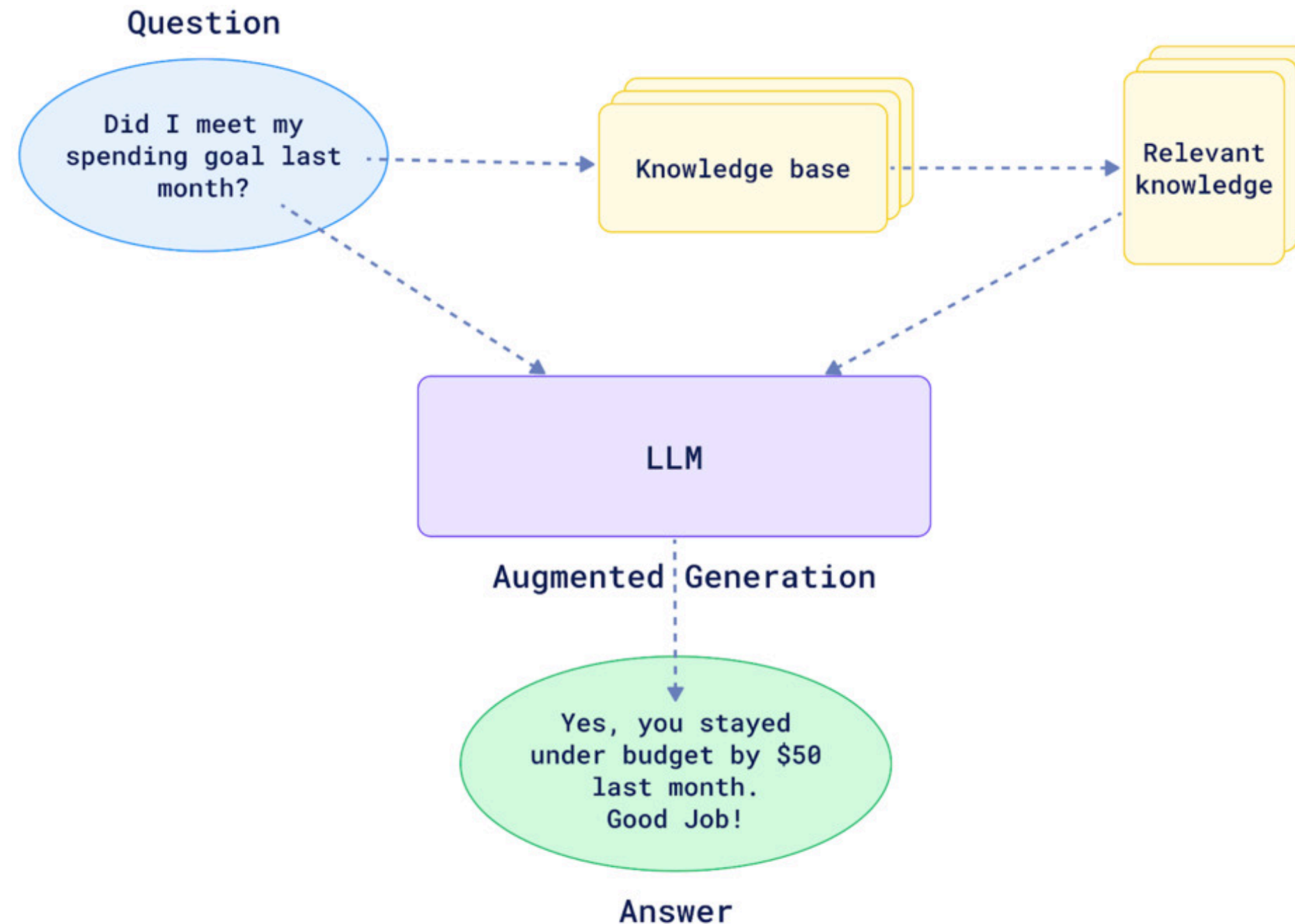
Add the retrieved knowledge to the AI's existing context to enhance understanding.

Generation

Create accurate, informed responses.

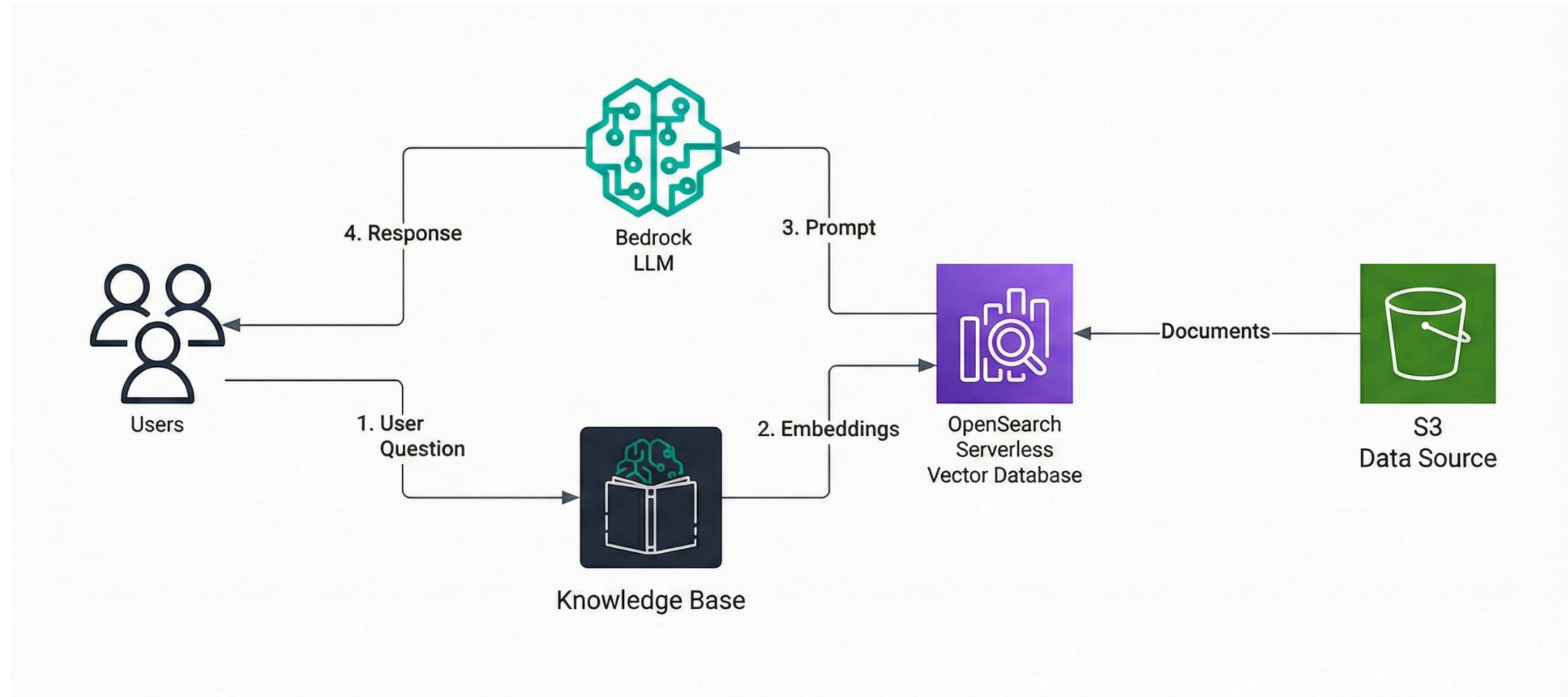
WHAT IS RAG?

Def: **Retrieval-Augmented Generation (RAG)** integrates an **external information** retrieval system into the process of generating responses by Large Language Models (LLMs) to improve the accuracy and relevance of the output by drawing on knowledge beyond its original pre-trained data.



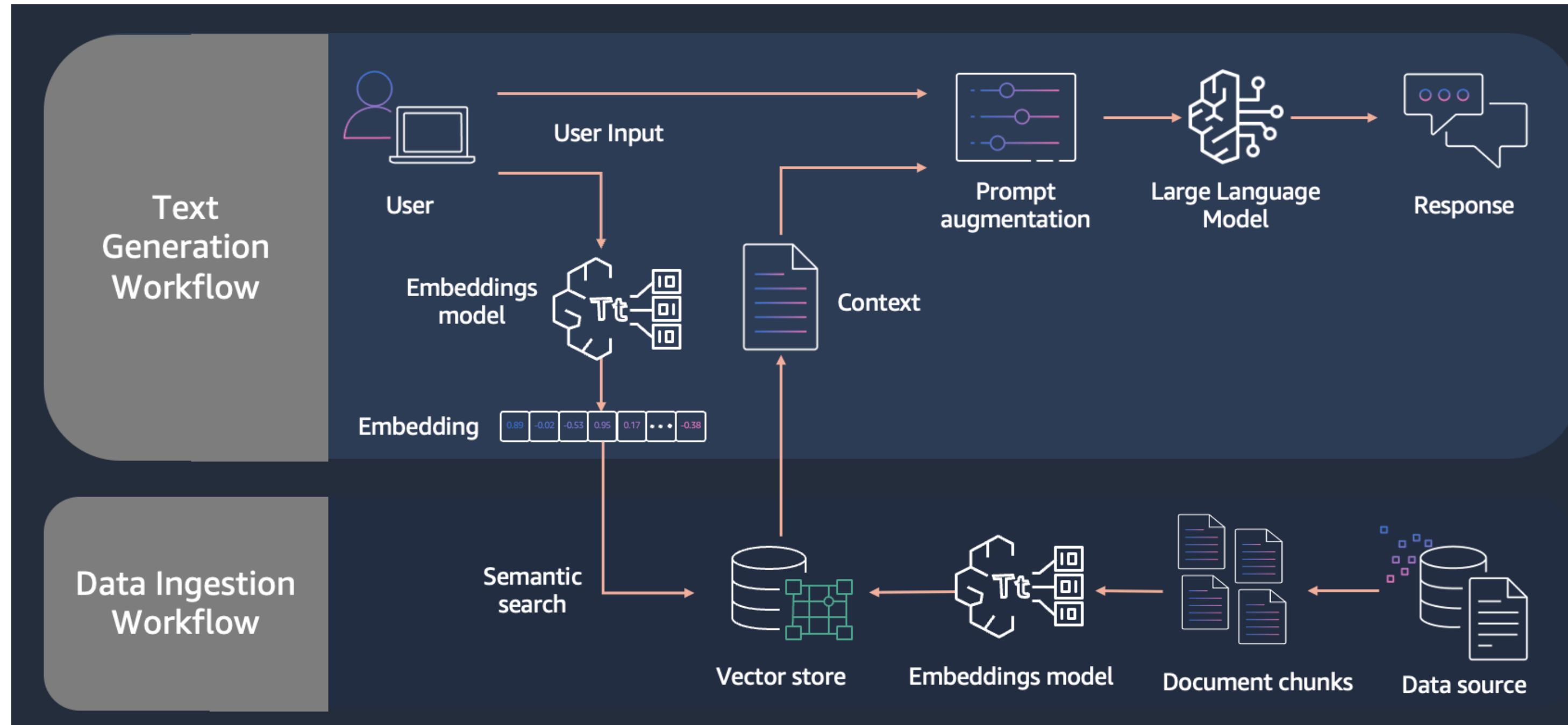
KNOWLEDGE BASE

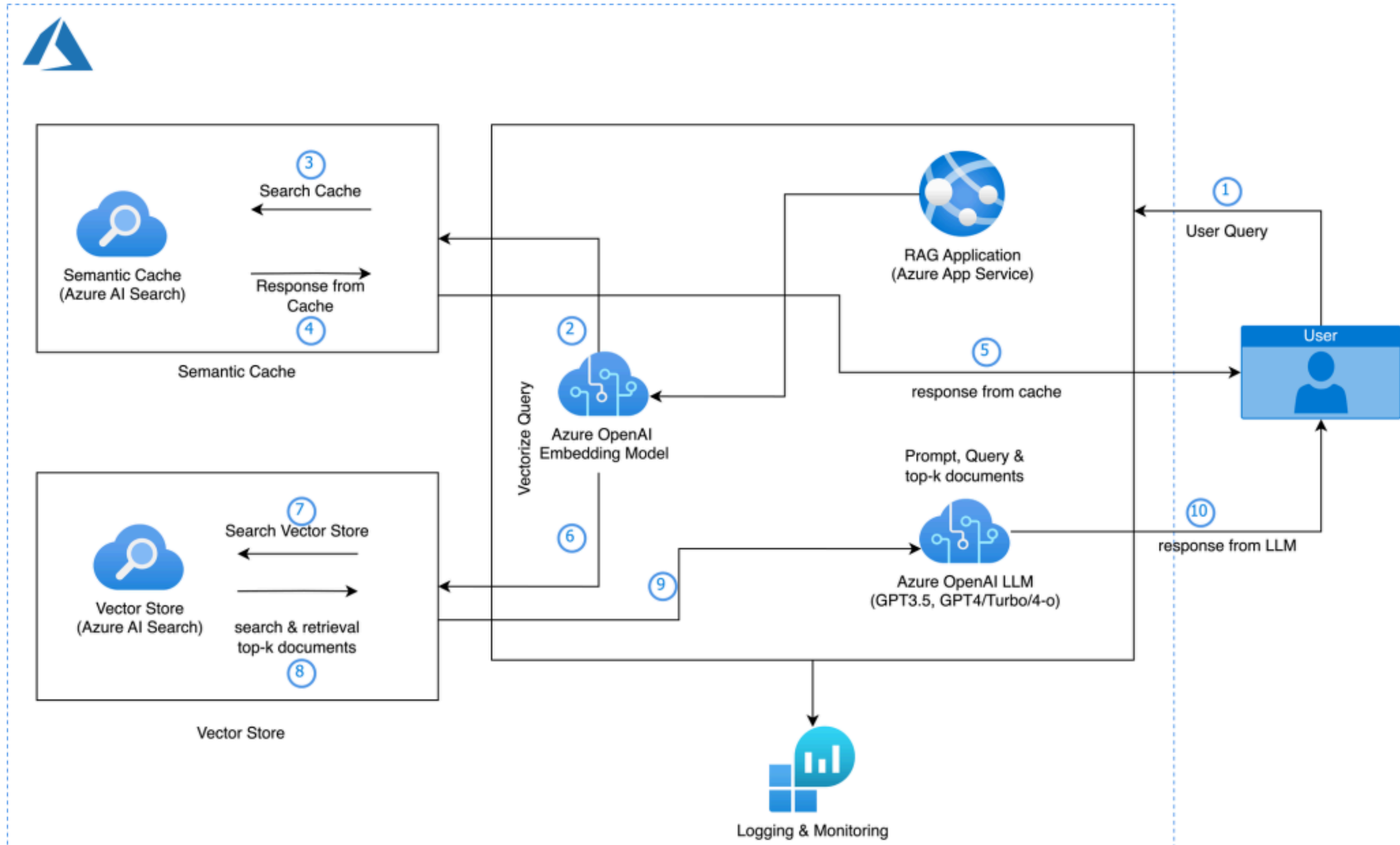
the knowledge source pipeline part of the RAG pattern.

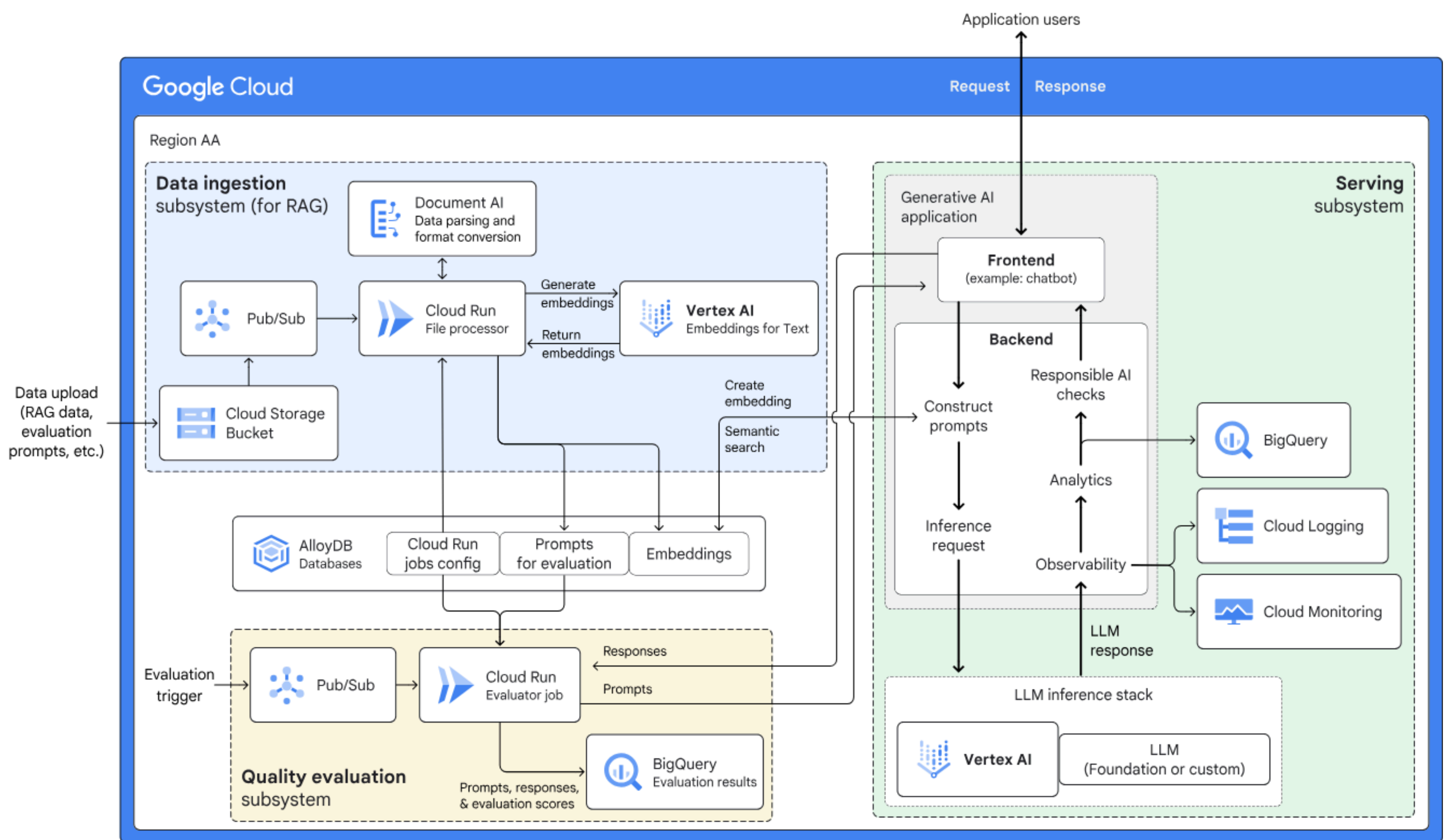


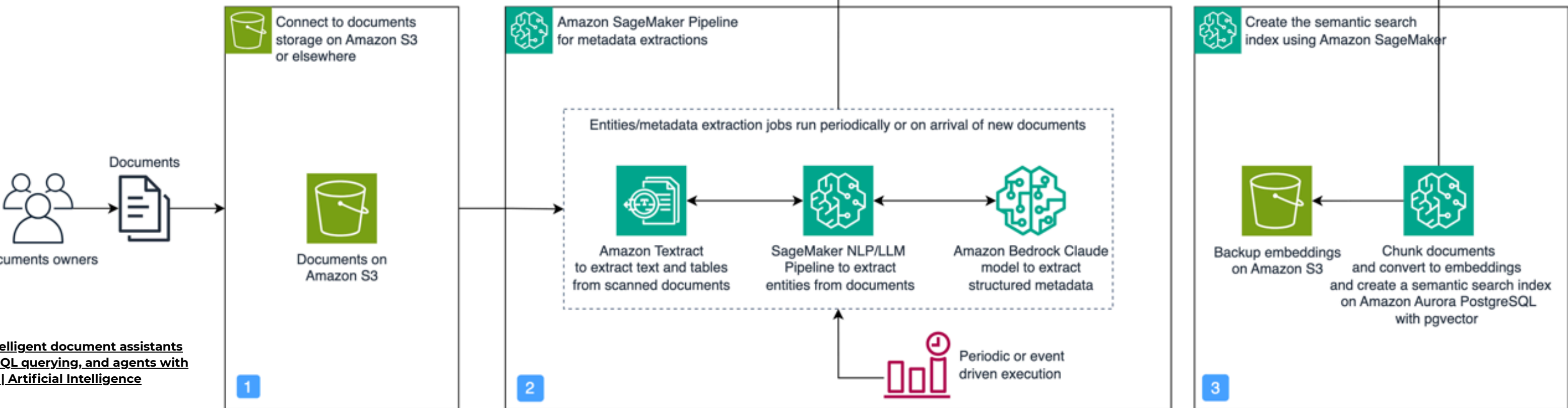
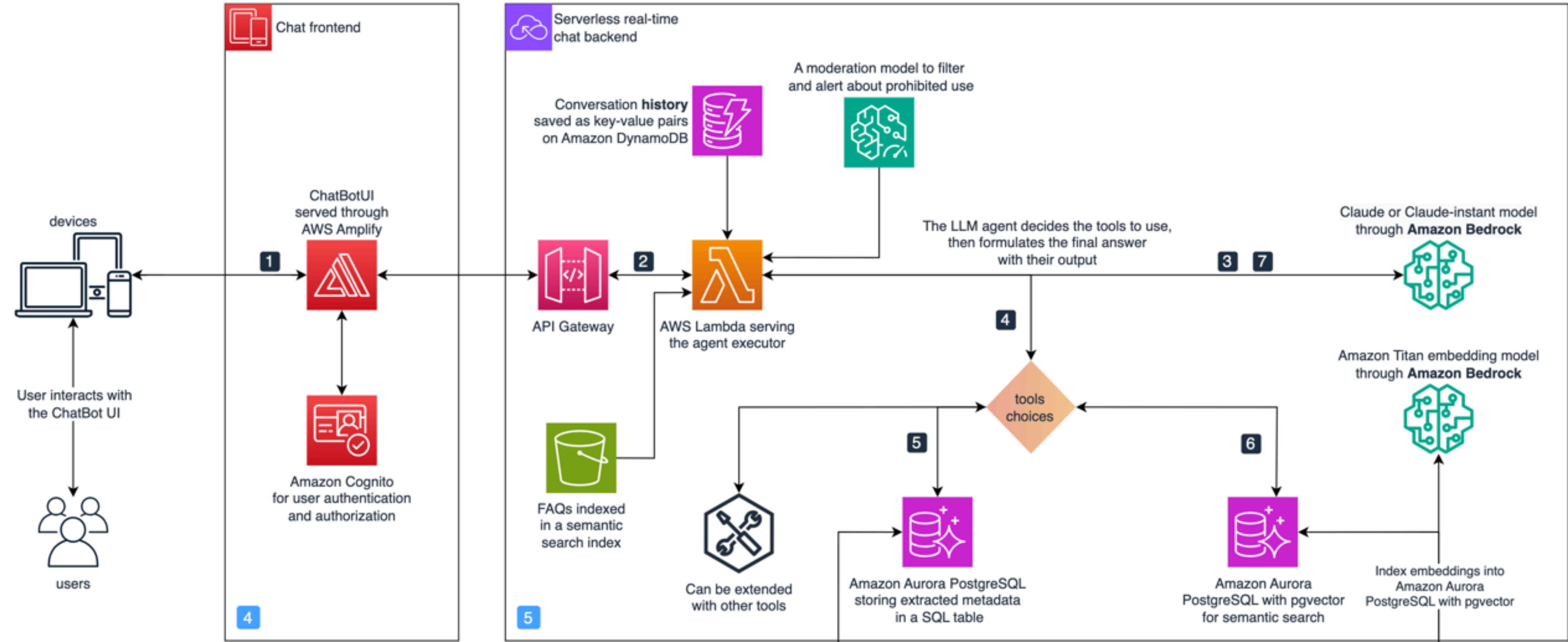
KNOWLEDGE BASE

the knowledge source pipeline part of the RAG pattern.



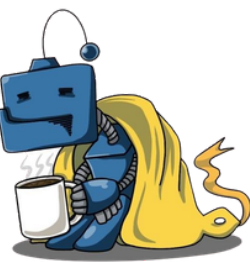




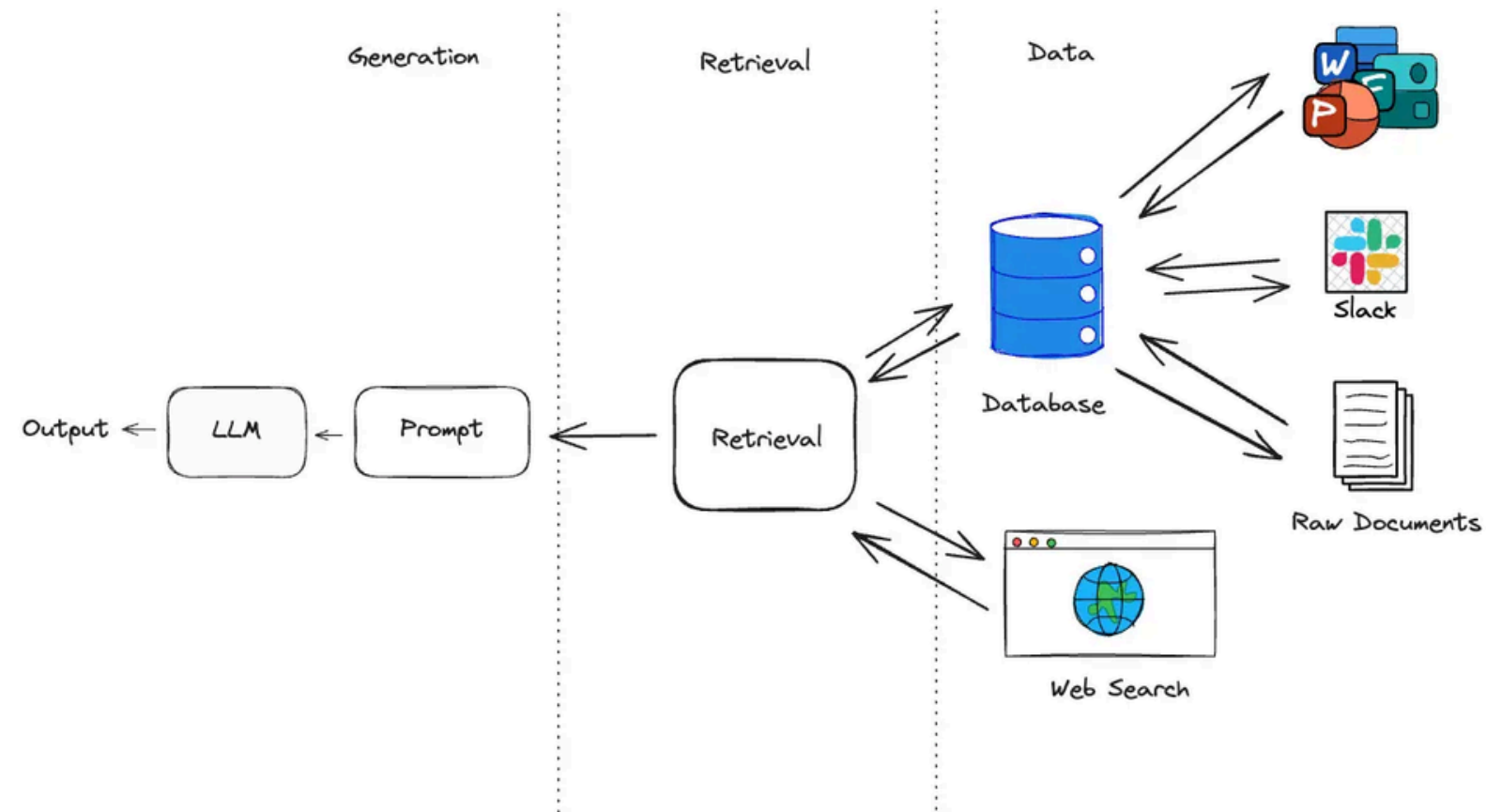


Boosting RAG-based intelligent document assistants using entity extraction, SQL querying, and agents with Amazon Bedrock | Artificial Intelligence

RAG COMPONENTS



RAG COMPONENTS



Data source

Text Data: Power point, Word, Excel, etc.
Unstructure Data: Image, Video
Structure Data: SQL Database, etc

Retriever

Text search, Vector search, Hybrid search, SQL query

LLM

Opensource, Comercial, etc.

DEEP DRIVE TO CHUNKING STRATEGIES

DATA SOURCE

Data source

Text Data: Power point, Word, Excel, etc.

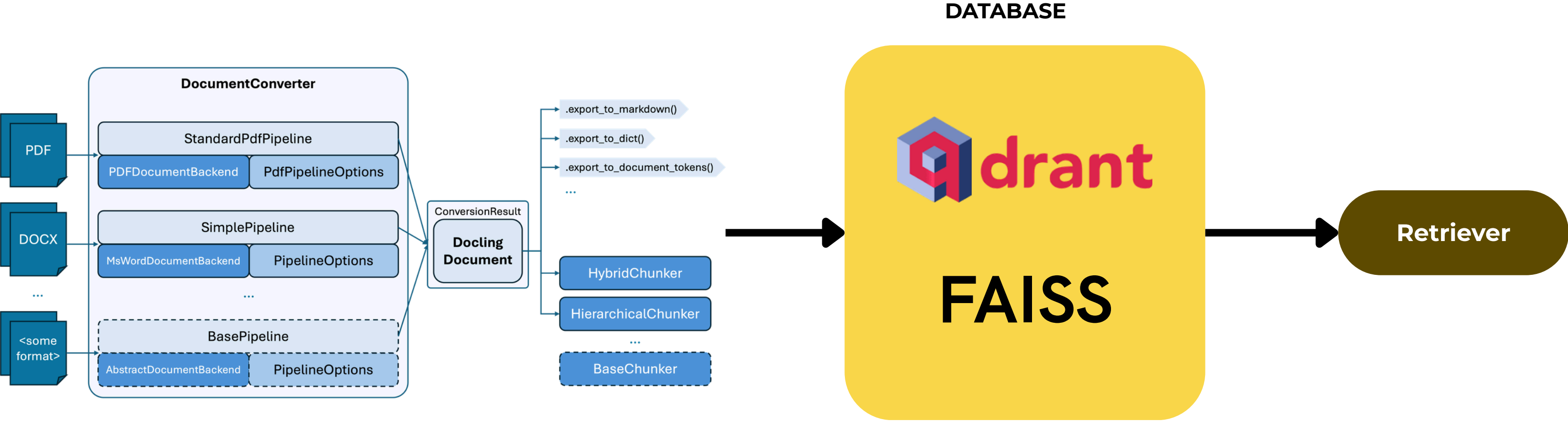
Unstructure Data: Image, Video

Structure Data: SQL Database, etc

Type	Description	Examples	Processing Method
Text Data	Raw textual documents	PDF, Word, PowerPoint, Markdown	Tokenization, Chunking, Embedding
Unstructured Data	Media without explicit schema	Images, Audio, Videos	OCR (Optical Character Recognition), Speech-to-Text, Captioning
Semi-Structured Data	Partially formatted	JSON, HTML, XML	Key-value parsing, metadata extraction
Structured Data	Organized with schema	SQL Database, CSV, Spreadsheets	SQL queries, table embedding

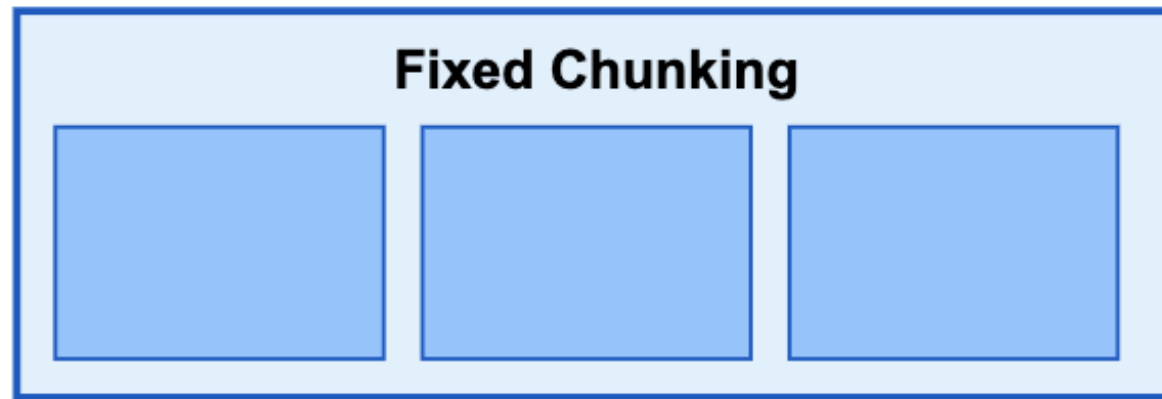


DATA SOURCE → RETRIEVER

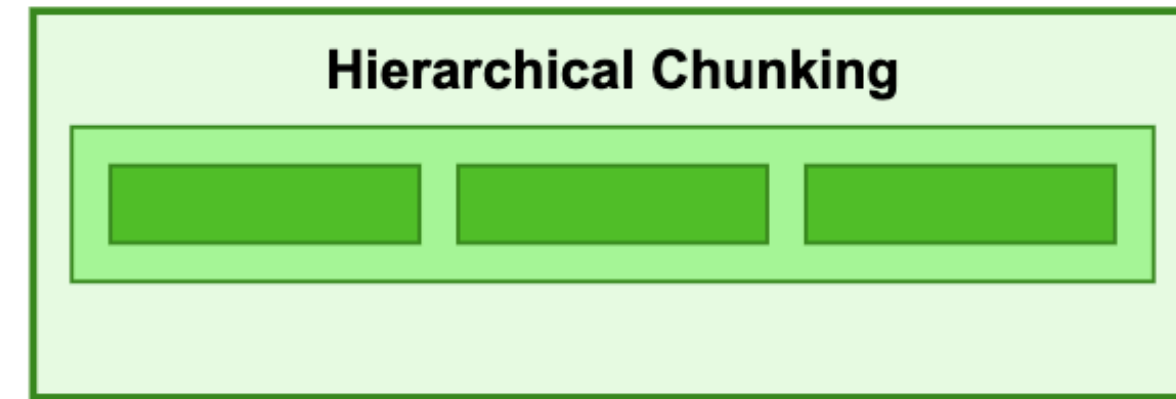


CHUNKING

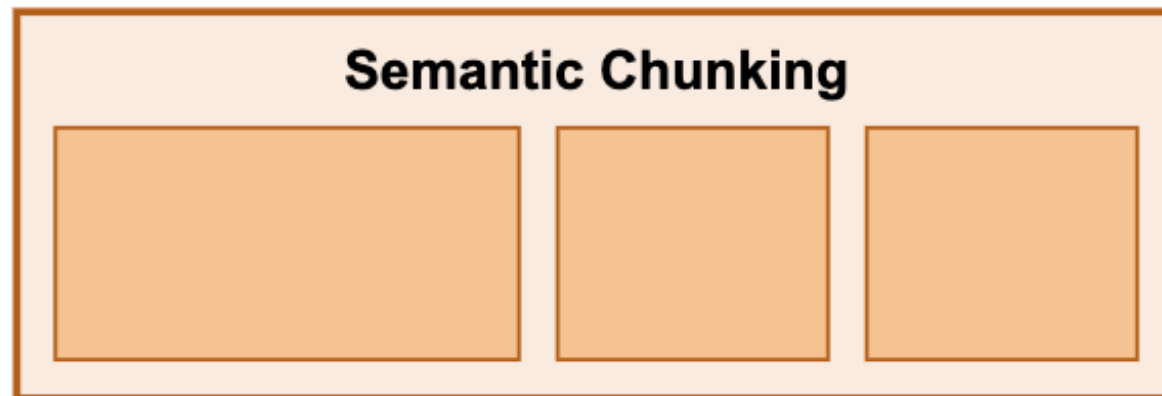
Chunking Strategies



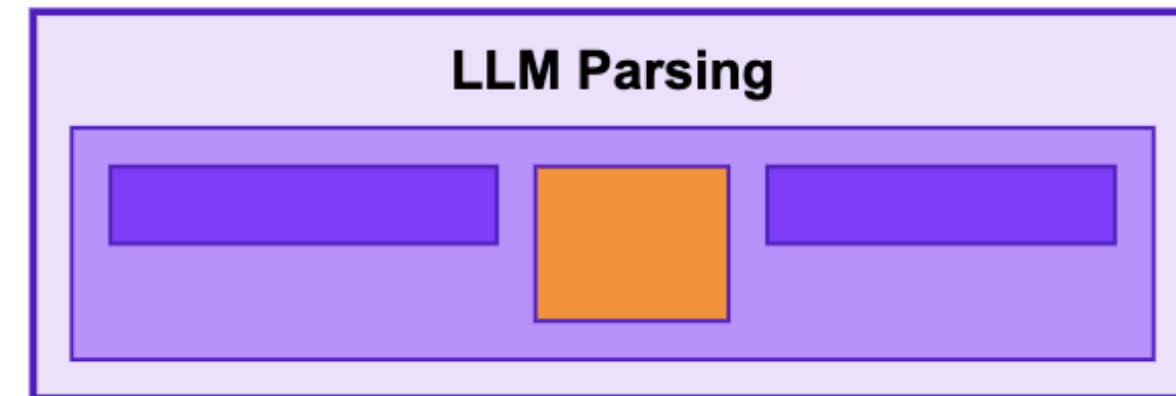
Fixed-size chunks with optional overlap



Parent and child chunks

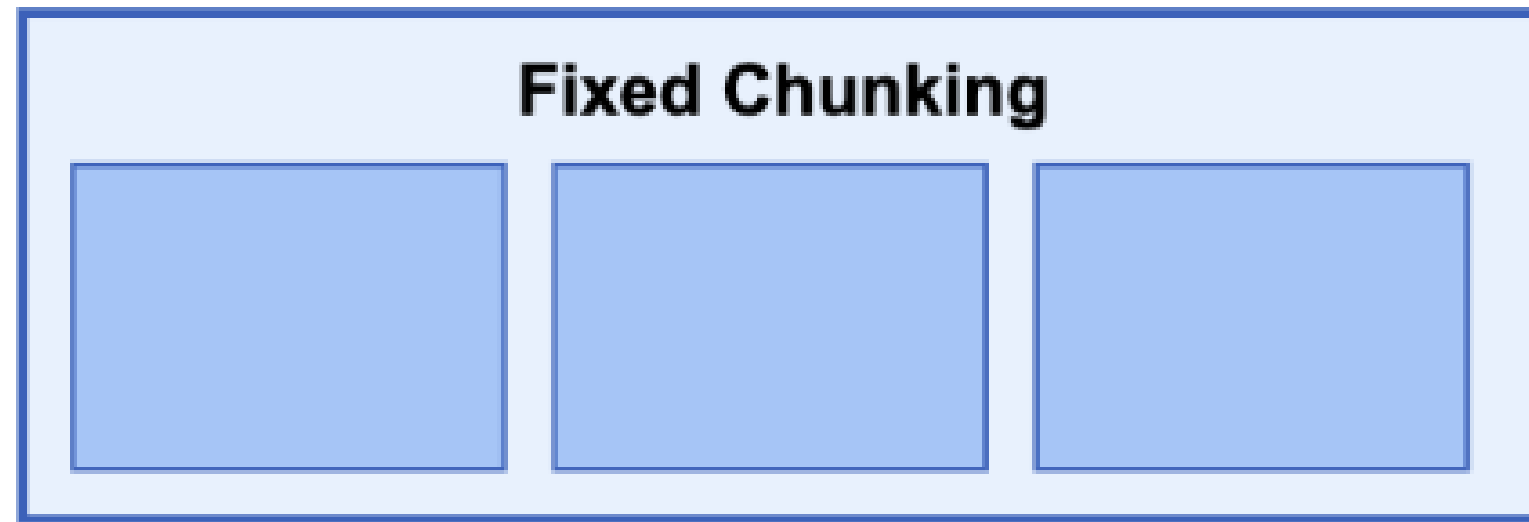


Chunks based on semantic similarity



Custom parsing with LLM instructions

CHUNKING



1. Fixed Chunking (แบ่งตามขนาดที่กำหนด)

คอนเซปต์: ตัดแบ่งเท่าๆ กันเป๊ะๆ เช่น ตัดทุกๆ 500 ตัวอักษร (อาจมีส่วนซ้อนทับกันนิดหน่อยหรือ Overlap เพื่อไม่ให้ประโยคขาดช่วง)

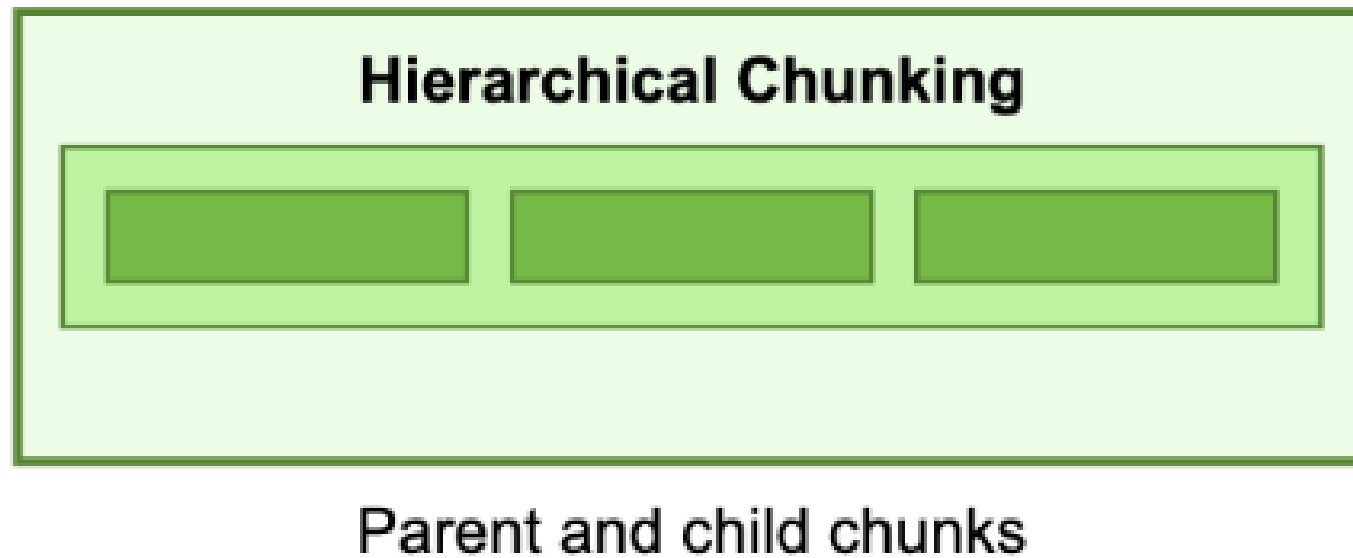
ข้อดี/เสีย: ง่ายและเร็วที่สุด แต่อาจตัดกลางประโยคหรือทำให้ใจความสำคัญขาดหายไป

สถานการณ์: เอกสาร "สัญญาเช่าหอพัก" หรือ "เงื่อนไขการใช้งานแอป (Terms of Service)"

- **วิธีทำ:** กำหนดว่าตัดทุกๆ 500 ตัวอักษร
- **ผลลัพธ์:**
 - **ก้อนที่ 1:** "...ผู้เช่าตกลงว่าจะไม่ส่งเสียงดังรบกวนผู้อื่น หากฝ่าฝืนผู้ให้เช่ามีสิทธิ์ [ตัดจบ]"
 - **ก้อนที่ 2:** "[ต่อ] บอกเลิกสัญญาได้ทันที และจะไม่คืนเงินประกัน..."
- **จุดสังเกต:** ในภาษาไทยมักมีปัญหา เพราะภาษาเราไม่มีเว้นวรรคระหว่างคำเหมือนภาษาอังกฤษ บางทีการตัดแบบนี้อาจจะไปตัดกลางคำ เช่น คำว่า "ตากลม" (ตาก-ลม) หรือตัดประโยคสำคัญขาดครึ่ง ทำให้ AI งงความหมายได้ครับ (แต่ข้อดีคือประมวลผลเร็วมาก)

CHUNKING

2. Hierarchical Chunking (แบ่งแบบลำดับชั้น)

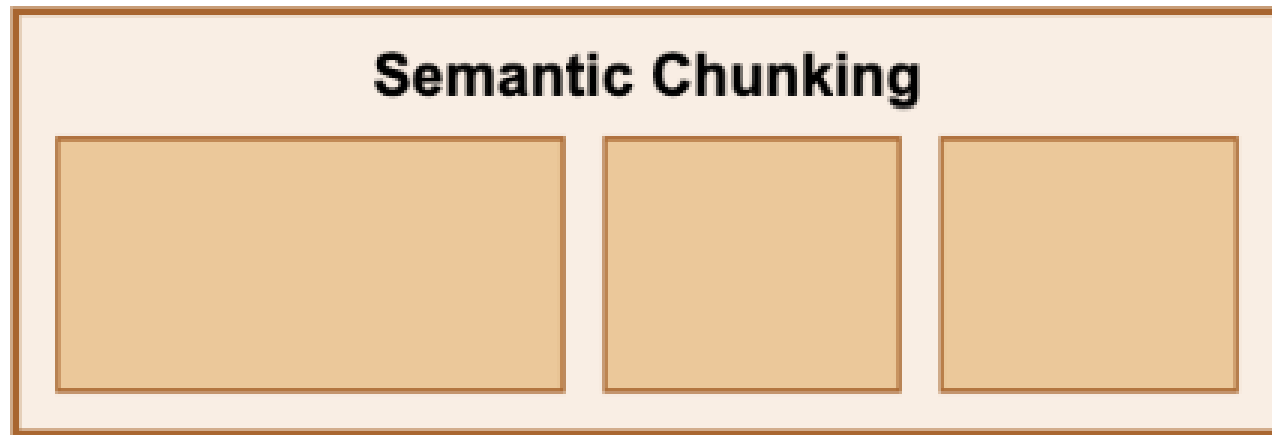


- **คอนเซปต์:** มีการเก็บข้อมูลเป็น 2 ระดับ คือ "ก้อนใหญ่" (Parent) ที่เก็บภาพรวม และ "ก้อนย่อย" (Child) ที่เจาะลึกเนื้อหา เวลาค้นหาเราจะหาจากก้อนย่อย แต่เวลา AI ตอบ มันจะดึงบริบทจากก้อนใหญ่ไปตอบด้วย
- **เปรียบเทียบ:** เหมือนการจัดหนังสือ ที่เราแยกเป็น "บทที่ 1" (Parent) และ "ย่อหน้าย่อยๆ ในบท" (Child) เวลาถามหาข้อมูลย่อยๆ เราก็จะรู้ด้วยว่ามันมาจากบทไหน ทำให้เข้าใจบริบทได้ดีกว่า

สถานการณ์: เอกสาร "คู่มือพนักงาน (Employee Handbook)" หรือ "กฎหมายแรงงาน"

- **วิธีทำ:** ระบบจะมองเป็นโครงสร้างใหญ่ (Parent) และย่อย (Child)
- **ผลลัพธ์:**
 - **Parent (ก้อนแม่):** "หมวดสวัสดิการรักษายาบาล" (เก็บภาพรวมเงื่อนไขทั้งหมด)
 - **Child (ก้อนลูก 1):** "วงเงินผู้ป่วยนอก (OPD) 2,000 บาท/ครั้ง"
 - **Child (ก้อนลูก 2):** "วงเงินทำฟัน 5,000 บาท/ปี"
- **จุดสังเกต:** เวลาถาม AI ว่า "ทำฟันเบิกได้เท่าไร" AI จะเจอก้อนลูก (5,000 บาท) แต่เวลามันตอบ มันจะดึงบริบทจากก้อนแม่มาด้วย ทำให้มันรู้ว่านี่คือสวัสดิการของบริษัท ไม่ใช่ราคาทำฟันทั่วไป

CHUNKING



Chunks based on semantic similarity

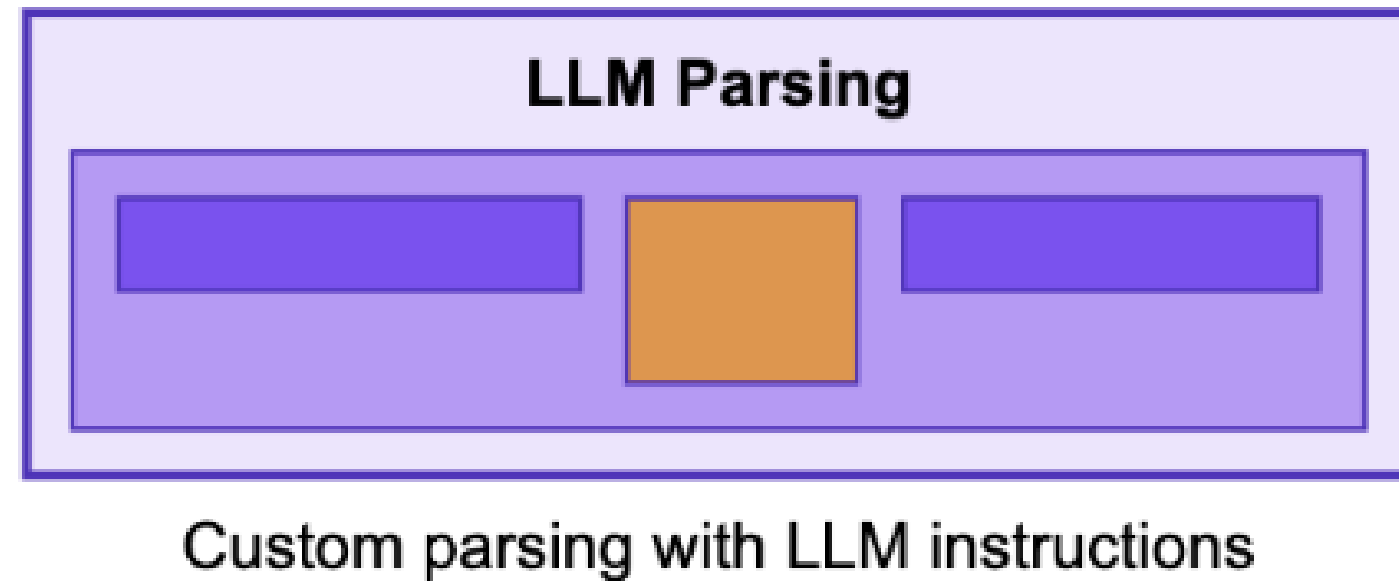
3. Semantic Chunking (แบ่งตามความหมาย)

- **คอนเซปต์:** ใช้ AI วิเคราะห์เนื้อหา ถ้าเรื่องเปลี่ยน ค่อยตัดแบ่ง (Semantic similarity) ดังนั้นแต่ละก้อนจะมีขนาดไม่เท่ากัน ขึ้นอยู่กับความยาวของเนื้อเรื่องนั้นๆ
- **เปรียบเทียบ:** เหมือนเราอ่านหนังสือแล้วค้นหน้าเมื่อ "จบบทสนทนา" หรือ "เปลี่ยนฉาก" วิธีนี้ทำให้ข้อมูลแต่ละชิ้นมีความหมายในตัวเองสมบูรณ์ ไม่ขาดตอน

สถานการณ์: บทความ "รีวิวเที่ยวเชียงใหม่" หรือ "ข่าวอาชญากรรมสรุปเหตุการณ์"

- **วิธีทำ:** ให้ AI อ่านไปเรื่อยๆ พอเปลี่ยนเรื่องค่อยตัด
- **ผลลัพธ์:**
 - **ก้อนที่ 1 (เรื่องกิน):** พูดถึงร้านข้าวซอยเสมอใจ รสชาติเป็นไง ราคาเท่าไร... (พอจบเรื่องกิน ตัดจบ)
 - **ก้อนที่ 2 (เรื่องที่พัก):** เริ่มพูดถึงโรงแรมย่านนิมมาน ห้องพัคดีไหม... (พอจบเรื่องที่พัก ตัดจบ)
 - **ก้อนที่ 3 (เรื่องการเดินทาง):** พูดถึงการเช่ารถแดง...
- **จุดสังเกต:** วิธีนี้ดีที่สุดสำหรับภาษาไทยที่เป็นการเล่าเรื่อง เพราะข้อมูลเรื่อง "ร้านอาหาร" จะไม่ไปปนกับ "โรงแรม" ทำให้เวลาถามเรื่องของกิน AI จะไม่เผลอหยิบข้อมูลโรงแรมมาตอบ

CHUNKING



4. LLM Parsing (ให้ AI ช่วยจัดระเบียบ)

- **คอนเซปต์:** ใช้โมเดลภาษาขนาดใหญ่ (LLM) อ่านข้อมูลแล้วสั่งให้มันดึงข้อมูลออกมาตามโครงสร้างที่เราต้องการ หรือให้มันสรุปและแบ่งหัวข้อให้เลย
- **เปรียบเทียบ:** เหมือนจ้างบรรณารักษ์เก่งๆ มานั่งอ่านหนังสือ แล้วสรุปย่อเขียนใส่การ์ดให้เราใหม่ตามที่เราสั่ง (เช่น "ช่วยดึงชื่อสินค้าและราคาออกมาหน่อย") วิธีนี้ฉลาดที่สุด แพงที่สุด แต่ได้ข้อมูลที่พร้อมใช้งานมากที่สุด

สถานการณ์: เอกสารที่รูปแบบสับสน เช่น "เรซูเม่ (Resume) ของผู้สมัครงาน" หรือ "ใบแจ้งหนี้ (Invoice) จากหลายซัพพลายเออร์"

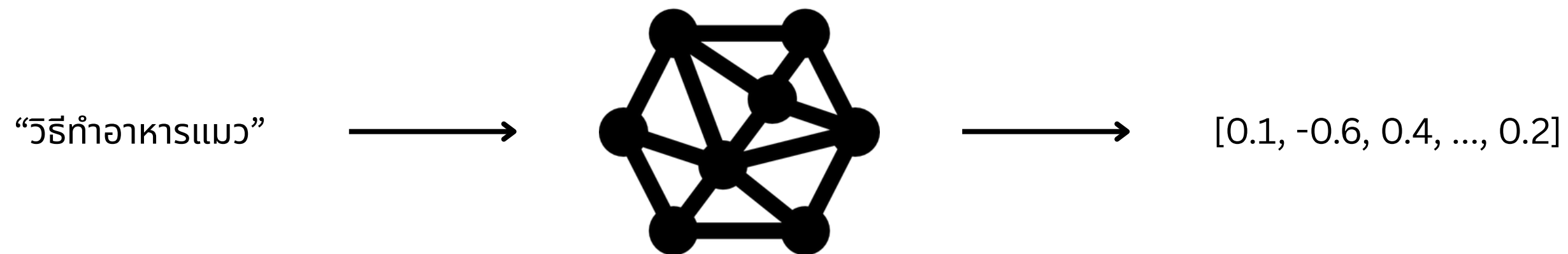
- **วิธีทำ:** เราสั่ง AI (LLM) ว่า "ช่วยดึง ชื่อ, เบอร์โทร, และประสบการณ์ทำงานล่าสุด ออกมาเป็นตารางให้หน่อย"
- **ผลลัพธ์:** ไม่ว่าเรซูเม่จะจัดหน้ามาแบบไหน (เขียนเรียงความ, ใส่ตาราง, กราฟิกเยอะ)
 - AI จะสกัดออกมาเป็น: { "ชื่อ": "สมชาย", "ตำแหน่ง": "วิศวกร", "สกิล": ["Python", "Java"]} }
- **จุดสังเกต:** เหมาะกับงานที่ต้องการความแม่นยำสูงมาก และต้องการข้อมูลที่มีโครงสร้างชัดเจนจากเอกสารที่ดูยากๆ แต่ค่าใช้จ่ายจะสูงที่สุด

Vectorization



QUERY VECTORIZATION

Def: **Query Vectorization** is the process where the user's new query is transformed into a **compatible query vector** using the exact same preprocessing and embedding techniques applied during the initial indexing phase.



User

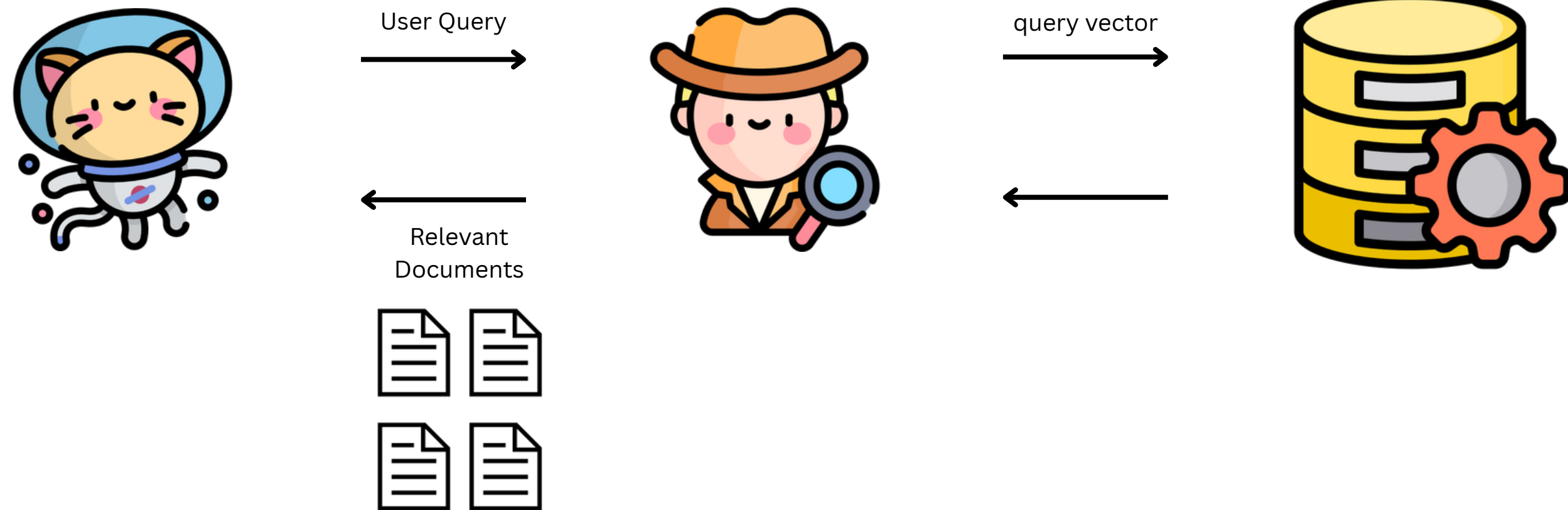


Embedding Model

RETRIEVAL OF RELEVANT DOCUMENTS

Recap

Def: **Retrieval of Relevant Documents** is the phase where the system utilizes **vector similarity techniques** (a fundamental concept in NLP) to quantify the resemblance between the query vector and the document vectors to find the most relevant information.



Natdhanai Praneenatthavee, AI engineer
เจ้าของเพจ ตื่นมาโค้ด python

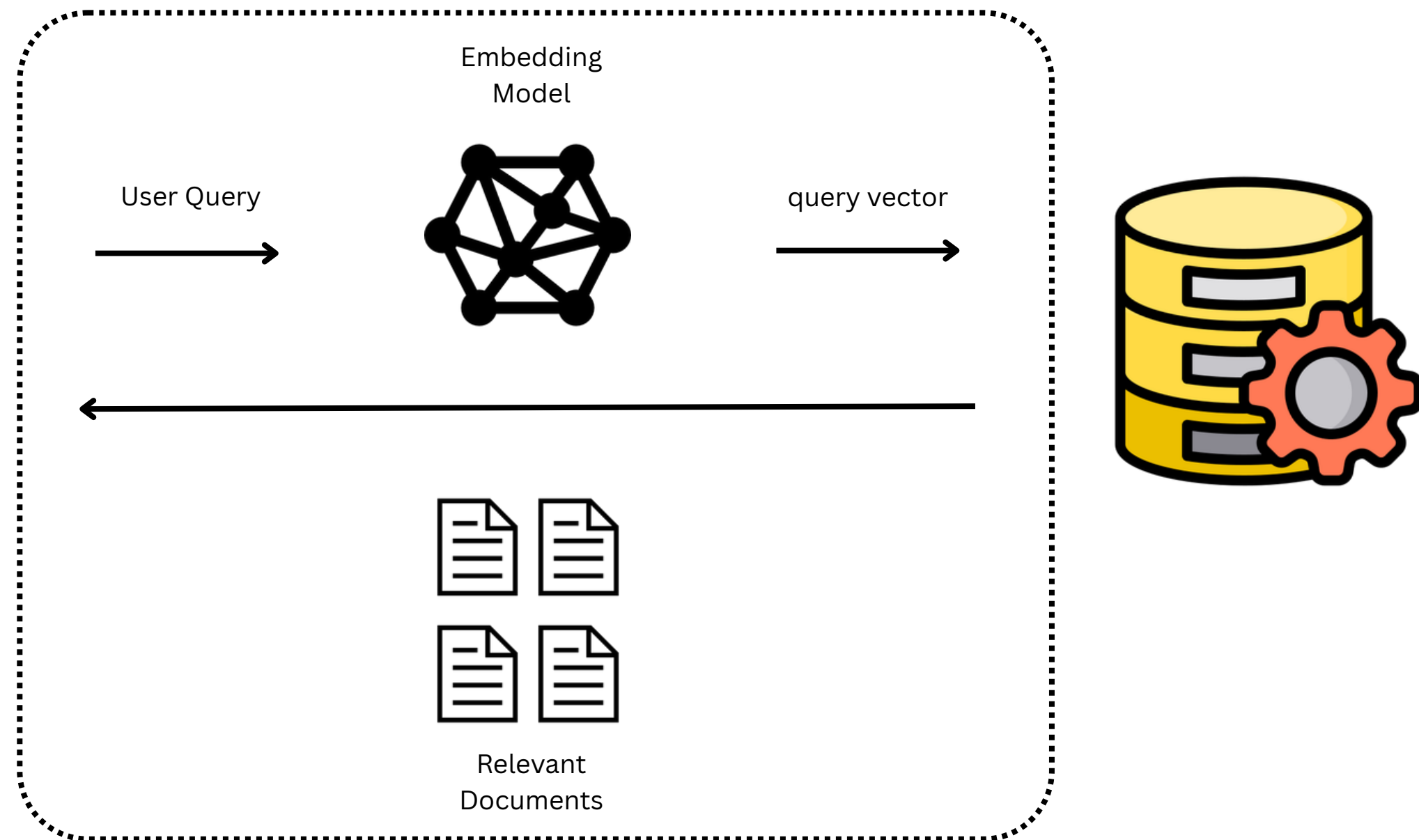
RETRIEVAL OF RELEVANT DOCUMENTS

Recap

Def: **Retrieval of Relevant Documents** is the phase where the system utilizes **vector similarity techniques** (a fundamental concept in NLP) to quantify the resemblance between the query vector and the document vectors to find the most relevant information.



Retriever



The Retriever



THE RETRIEVER

Def: **The Retriever** is the core component of RAG that uses **similarity search** to scan through a vast knowledge base of vector embeddings and **pull out the most relevant vectors/documents** to help answer the user query.



Natdhanai Praneenatthavee, AI engineer
เจ้าของเพจ ตื่นมาโค้ด python

Vector Similarity Strategies

VECTOR SIMILARITY STRATEGIES

A yellow sticky note with a red pushpin at the top left corner. The word "Recap" is written in black cursive script on the note.

Sparse Vector Representations

Def: **Sparse Vector Representations** are characterized by high dimensionality with most elements being zero, typically created by classic approaches like **keyword search** (e.g., TF-IDF or BM25) which prioritize exact word matches and word rarity.

Dense Vector Embeddings

Def: **Dense Vector Embeddings** are compact numerical representations encoded by large language models (like BERT) that **capture semantic meaning**, allowing the retriever to match information based on conceptual understanding rather than just keywords, often using metrics like cosine similarity.

Hybrid Search

Def: **Hybrid Search** is a strategy that **combines the strengths of different techniques** (such as keyword search and semantic vector search) to provide higher quality, more comprehensive results by compensating for the weaknesses of each individual method.

KEYWORD SEARCH

Def: **Keyword Search (or Sparse Vector Representations)** are characterized by high dimensionality with most elements being zero, typically created by classic approaches like **keyword search** (e.g., TF-IDF or BM25) which prioritize exact word matches and word rarity.

	about	bird	heard	is	the	word	you
About the bird, the bird, bird bird bird	1	5	0	0	2	0	0
You heard about the bird	1	1	1	0	1	0	1
The bird is the word	0	1	0	1	2	1	0

Sparse Vector Representations

KEYWORD SEARCH: TF-IDF

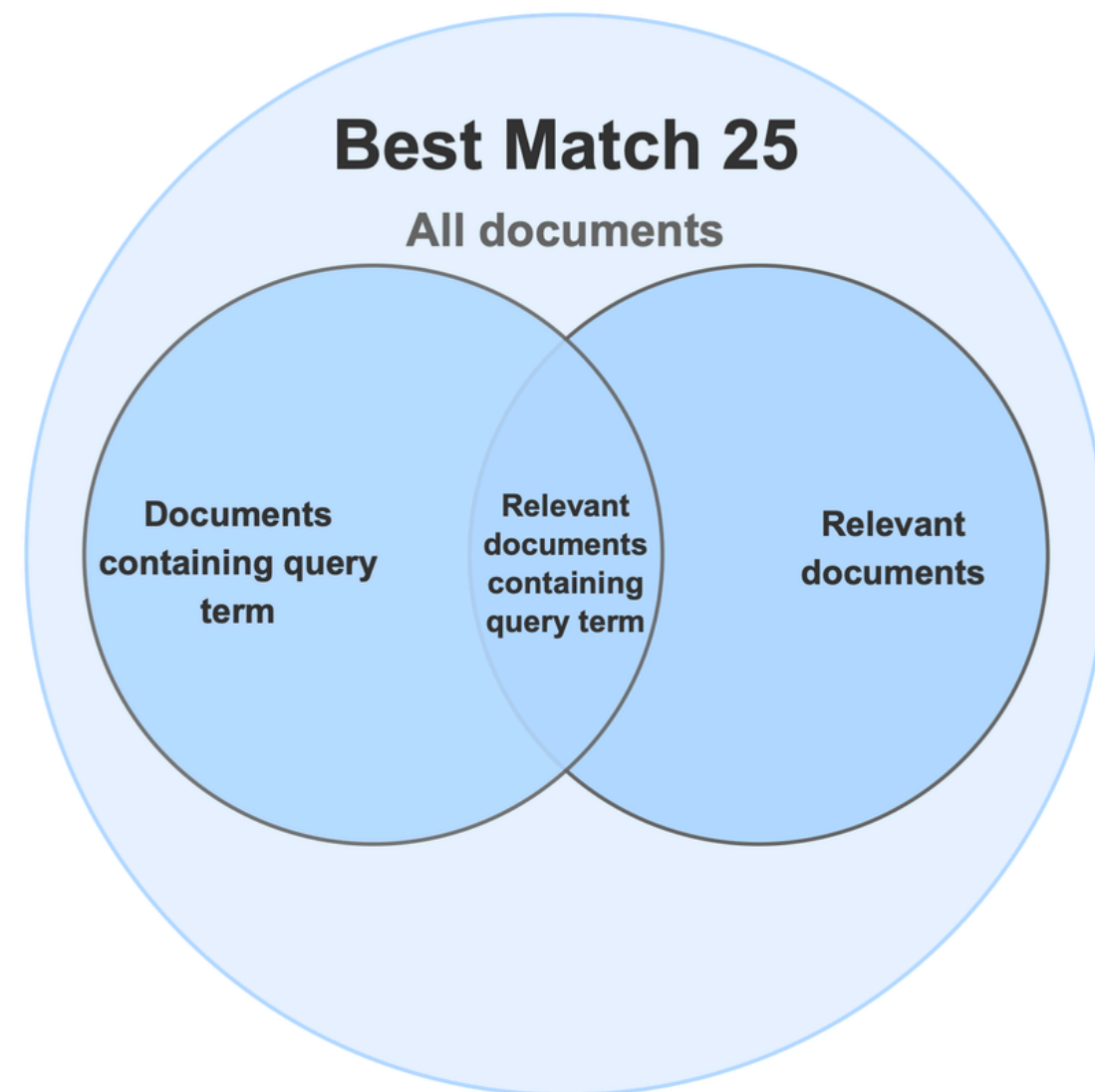
Def: **TF-IDF (Term Frequency-Inverse Document Frequency)** is a **numerical statistic** that quantifies the **importance of a word** in a specific document relative to an entire corpus by multiplying two factors: **Term Frequency (TF)**, which emphasizes words frequent within the document, and **Inverse Document Frequency (IDF)**, which penalizes words common across all documents.

Word	TF		IDF	TF*IDF	
	A	B		A	B
The	1/7	1/7	$\log(2/2) = 0$	0	0
Car	1/7	0	$\log(2/1) = 0.3$	0.043	0
Truck	0	1/7	$\log(2/1) = 0.3$	0	0.043
Is	1/7	1/7	$\log(2/2) = 0$	0	0
Driven	1/7	1/7	$\log(2/2) = 0$	0	0
On	1/7	1/7	$\log(2/2) = 0$	0	0
The	1/7	1/7	$\log(2/2) = 0$	0	0
Road	1/7	0	$\log(2/1) = 0.3$	0.043	0
Highway	0	1/7	$\log(2/1) = 0.3$	0	0.043

A: The car is driven on the road.
B: The truck is driven in the highway.

TF-IDF

KEYWORD SEARCH: BM25



BM25 (Best Matching 25) คืออัลกอริทึมการจัดอันดับ (ranking function)

องค์ประกอบสำคัญของ BM25:

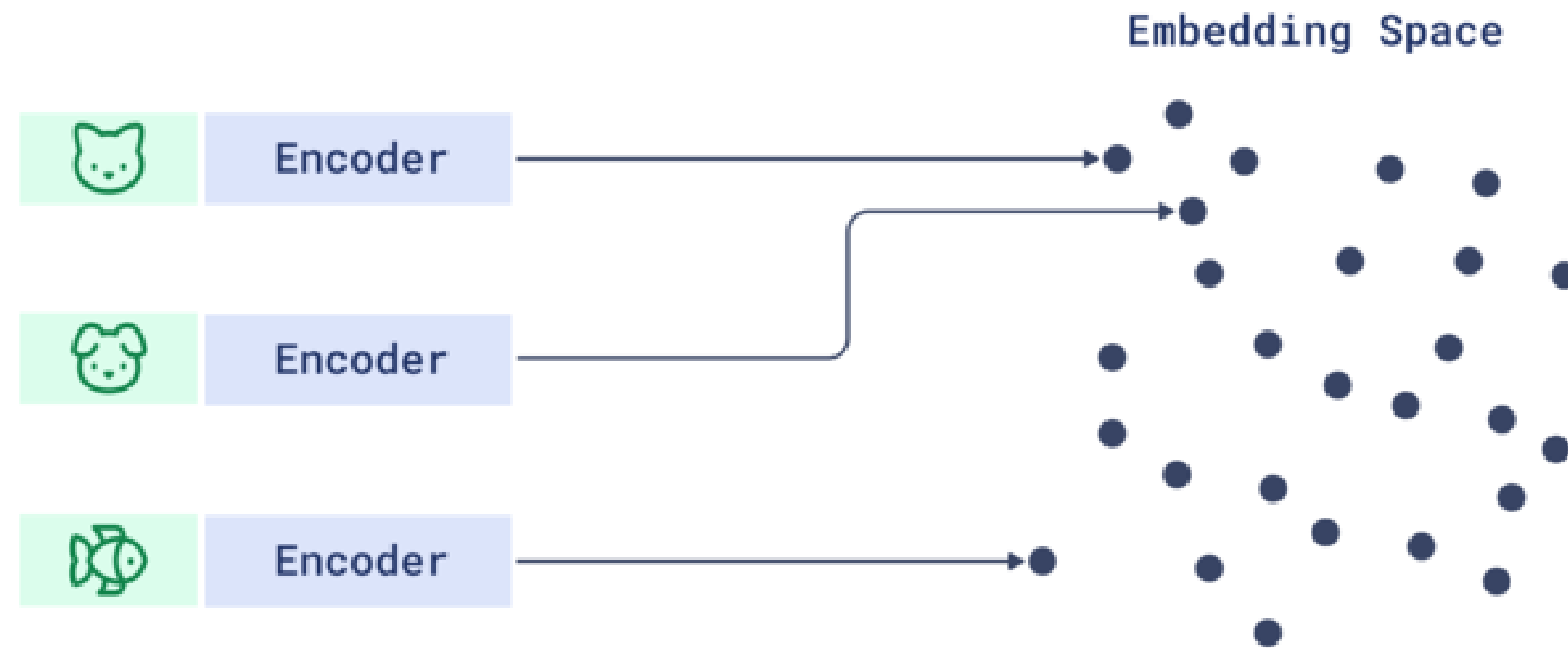
- ความถี่ของคำ (Term Frequency - TF): ยิ่งคำค้นหาปรากฏในเอกสารบ่อยครั้ง คะแนนยิ่งสูง แต่มีจุดอิ่มตัว (saturation) เพื่อไม่ให้คำที่ซ้ำกันมากเกินไปมีอิทธิพลสูงเกินไป.
- ความหายากของคำ (Inverse Document Frequency - IDF): ให้คะแนนสูงกับคำที่หายากในชุดเอกสารทั้งหมด และให้คะแนนต่ำกับคำที่พบบ่อย.
- การปรับตามความยาวเอกสาร (Document Length Normalization): ปรับคะแนนเพื่อให้เอกสารสั้นและยาวมีความเป็นธรรม ไม่ให้เอกสารยาวเกินไปได้เปรียบโดยไม่จำเป็น.

ทำไมถึงสำคัญกว่า TF-IDF:

BM25 ปรับปรุงข้อจำกัดของ TF-IDF โดยการเพิ่มการปรับตามความยาวเอกสาร และใช้ฟังก์ชันที่ซับซ้อนกว่าในการจัดการความถี่คำ ทำให้ผลลัพธ์การค้นหามีความแม่นยำและเป็นธรรมชาติมากขึ้น.

SEMANTIC SEARCH

Def: **Semantic Search (or Dense Vector Embeddings)** are compact numerical representations encoded by large language models (like BERT) that **capture semantic meaning**, allowing the retriever to match information based on conceptual understanding rather than just keywords, often using metrics like cosine similarity.



This is how vector similarity works

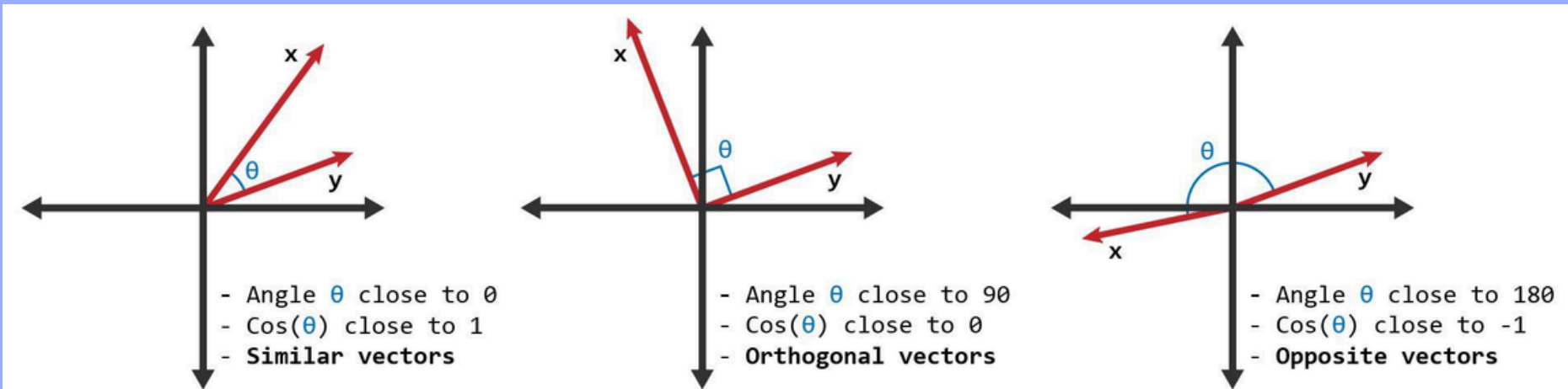
SEMANTIC SEARCH: COSINE SIMILARITY

Def: **Cosine Similarity** is a widely used similarity metric that determines how similar two non-zero vectors are based on the **cosine of the angle (θ) between them**, ignoring their magnitude (length). Scores range from 1 (exact same direction) to -1 (exactly opposite direction), making it especially effective for comparing data points in **high-dimensional** and **sparse datasets**, such as those in NLP (e.g., word embeddings or TF-IDF vectors).

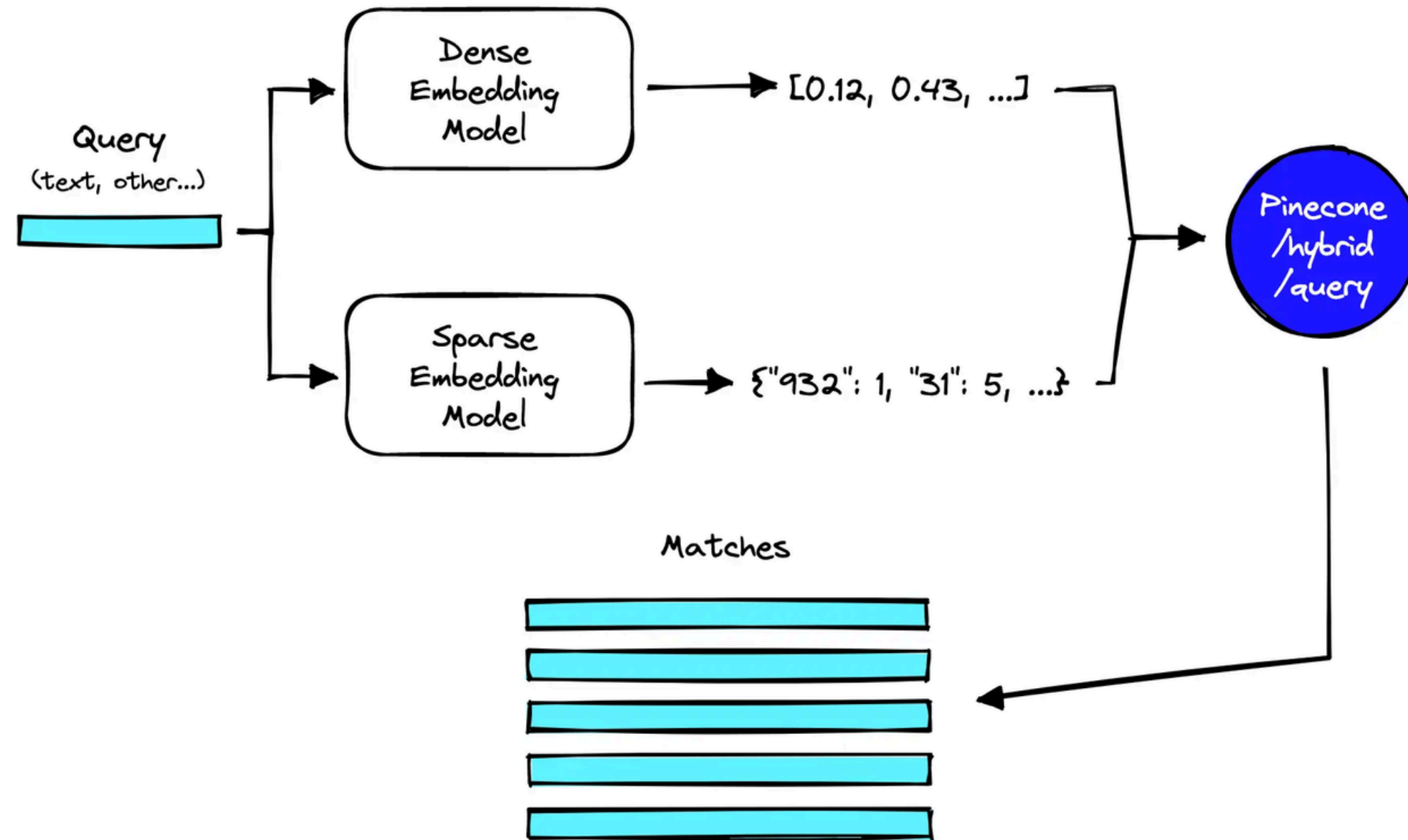
Formula

$$\text{similarity}(A, B) = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2 \sum_{i=1}^n B_i^2}}$$

Interpretation



HYBRID SEARCH



Hybrid Search คืออะไร?

Hybrid Search คือการค้นหาที่ "ลูกผสม" ระหว่าง 2 เทคนิค เพื่อให้ได้ผลลัพธ์ที่แม่นยำที่สุด ได้แก่:

1. **Keyword Search (Sparse Vectors):** ใช้ระบบ BM25 เน้นการค้นหา "คำที่ตรงกันเป๊ะๆ" (Exact Match) เหมาะกับชื่อเฉพาะ, รหัสสินค้า
2. **Vector Search (Dense Vectors):** ใช้ Machine Learning เน้นการค้นหาจาก "ความหมายและบริบท" (Context) เหมาะกับประโยคคำถาม หรือคำที่มีความหมายใกล้เคียงกัน

HYBRID SEARCH: RRF

เอกสาร	คะแนนจาก Keyword (BM25)	คะแนนจาก Vector (Dense)	คะแนนรวม (Hybrid Score)
B	อันดับ 2 ($1/2 = 0.5$)	อันดับ 1 ($1/1 = 1.0$)	$0.5 + 1.0 = \mathbf{1.5}$ (ชนะเลิศ)
A	อันดับ 1 ($1/1 = 1.0$)	อันดับ 3 ($1/3 \approx 0.33$)	$1.0 + 0.33 = \mathbf{1.33}$ (รองชนะเลิศ)
C	อันดับ 3 ($1/3 \approx 0.33$)	อันดับ 2 ($1/2 = 0.5$)	$0.33 + 0.5 = \mathbf{0.83}$

The Generator

THE GENERATOR

Def: **The Generator** is the component (typically an LLM like GPT, BART, or T5) responsible for producing the **final answer** by synthesizing and expressing the retrieved information in natural language.



Natdhanai Praneenatthavee, AI engineer
เจ้าของเพจ ตื่นมาโค้ด python

GENERATOR INPUT

Def: The Generator takes two key inputs: **the original query** (or question) and the **top relevant documents/passages** that the Retriever identified as potentially containing the answer.



question



relevant documents

GENERATOR PROCESS

Def: The Generator synthesizes the retrieved content and the user query to **produce a fluent, non-hallucinated response**, ensuring the final output is both knowledgeable and contextualized.



Natdhanai Praneenatthavee, AI engineer
เจ้าของเพจ ตื่นมาโค้ด python

MODEL SELECTION

Model selection

- Modality
 - Text
 - Text&Vision
 - Image
 - Embedding
- Capability
 - Tasks
 - Reasoning
 - Chat
 - Customize
- Speed
 - Token/Sec
 - Streaming
- Hosting
 - API
 - Server
 - Local
- Cost
 - Cost/Token
 - Infra

Model From

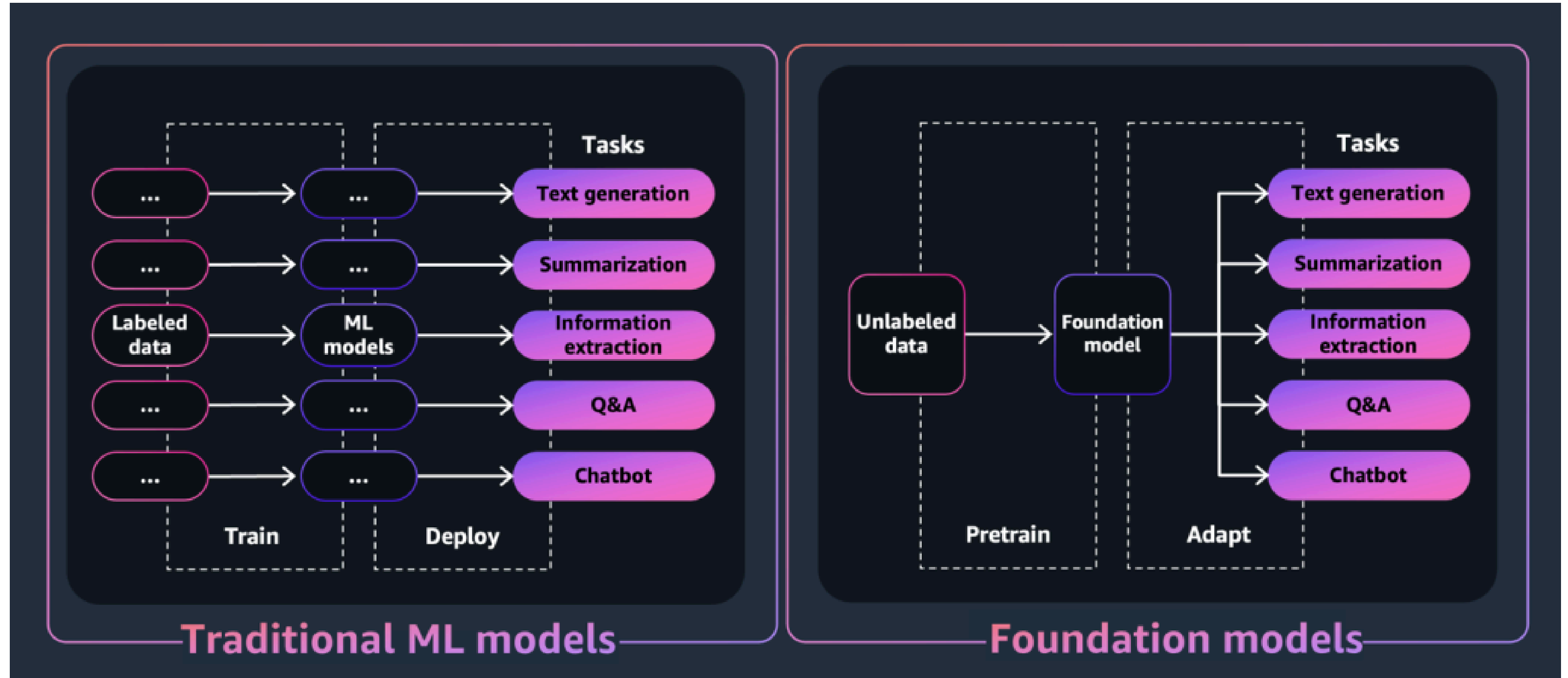
- Foundation models
- Amazon Bedrock Marketplace models
- Customized foundation models
- Imported foundation models
- Prompt routers
- Models that have purchased Provisioned Throughput

Evaluations

1. Automatic model evaluation jobs
2. Model evaluation jobs that use human workers
3. Model evaluation jobs that use a judge model
4. Overview of RAG evaluations that use Large Language Models (LLMs)

Natdhanai Praneenatthavee, AI engineer
เจ้าของเพจ ตื่นมาโค้ด python

TRADITIONAL VS FOUNDATION MODEL



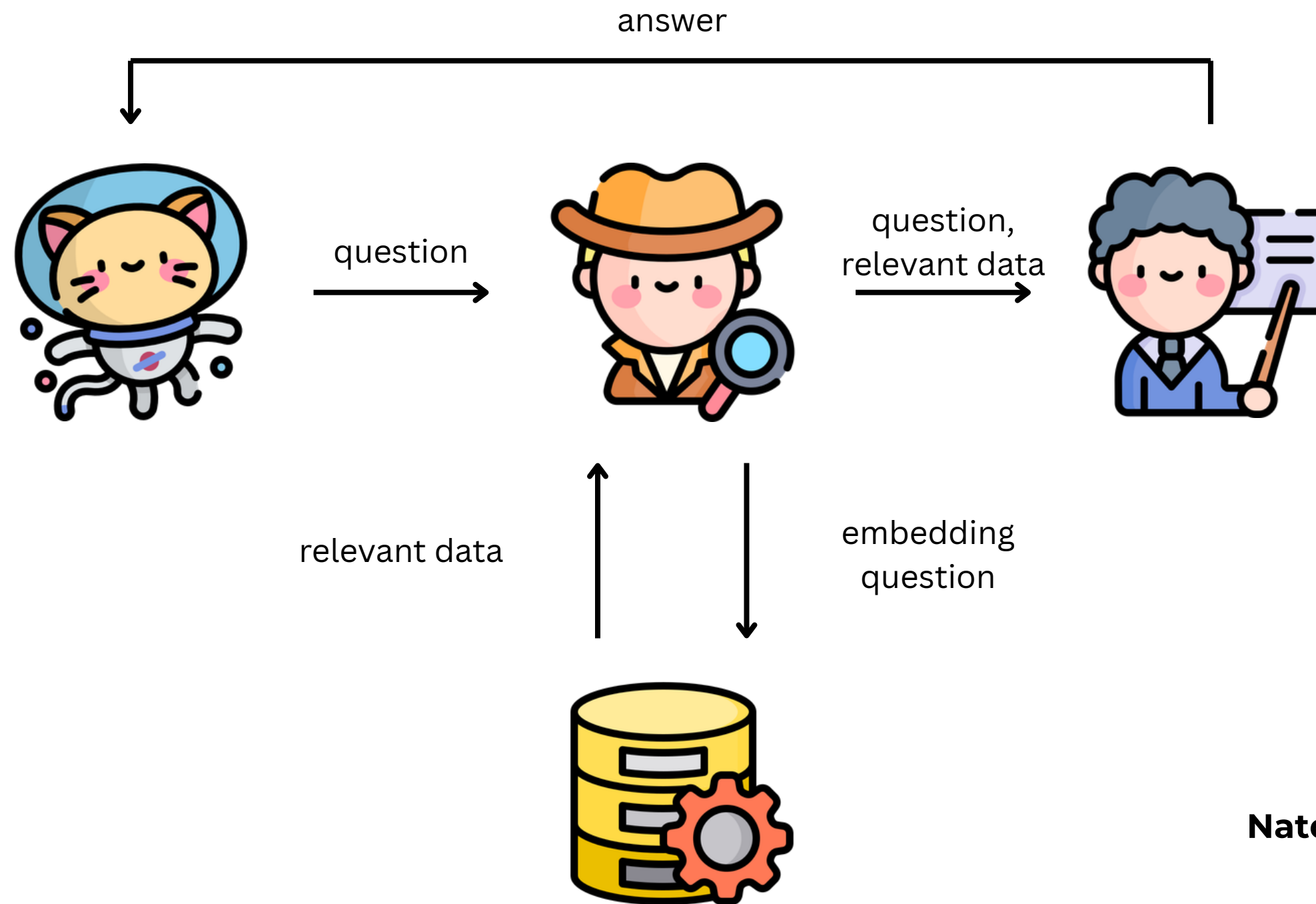
Natdhanai Praneenatthavee, AI engineer
เจ้าของเพจ ตื่นมาโค้ด python

Summary

RAG ARCHITECTURE (OVERALL)



Def: The **entire RAG architecture** is the integrated system where the **Retriever's output (relevant passages)** feeds directly into the **Generator**, enabling the LLM to produce a final, context-aware response.



Natdhanai Praneenatthavee, AI engineer
เจ้าของเพจ ตื่นมาโค้ด python

QUERY INTENT CLASSIFICATION



Def: Query Classification (or **Intent Classification**) is a preprocessing step that **determines the user's goal** (e.g., profile, schedule, purchase) before the retrieval process.

Goal: To differentiate a user's question, such as "Who is Dr. Hasan?" (Profile Intent) from "When is Dr. Hasan available?" (Schedule Intent).

Core Challenge: Basic keyword matching is often unreliable, necessitating **advanced NLP techniques** to accurately capture the user's intent from the context.

TECHNIQUES FOR INTENT CLASSIFICATION

- **NER (Named Entity Recognition):** Extracts structured information (e.g., PERSON: Dr. Hasan, DATE: Friday) to deduce the query's purpose.
- **Dependency Parsing:** Analyzes the grammatical structure (e.g., the ROOT word) to understand complex query intent.
- **LLM Few-shot Prompting:** Uses a powerful LLM (like GPT-4) with a few examples to classify the intent with high, context-aware accuracy.
- **Fine-tuned Transformer:** Training models like BERT for fast, production-level query classification and best accuracy (high resource requirement).

INTENT CLASSIFICATION MEETS THE RETRIEVER

HOW INTENT CLASSIFICATION OPTIMIZES RAG

- **Indexing Optimization (Chunking + Metadata)**
 - **Action:** Documents are broken into meaningful, smaller chunks (e.g., 'profile' data vs. 'schedule' data).
 - **Result:** Each chunk is tagged with corresponding **metadata** (e.g., {"type": "schedule"}) before being stored as a vector.
- **Retriever Optimization (Metadata Filtering)**
 - **Action:** The identified Query Intent is used to create a **metadata filter**.
 - **Result:** The filter is applied to the vector database **before** computing vector similarity search, ensuring only the relevant subset of vectors is processed.

INTENT CLASSIFICATION MEETS THE RETRIEVER

The Hybrid Power Play: Intent + Metadata + Search

This advanced approach enhances the Vector Similarity Strategies:

- **Enhancing Dense Vector Embeddings:**
 - The intent directs the system to perform a **Cosine Similarity Search *only*** within the filtered subset of vectors that match the required metadata.
- **Creating Optimized Hybrid Search 🔥:**
 - The intent/metadata acts as an **initial, highly efficient filter**, which is then followed by the standard vector or keyword search on the reduced set of candidates.
- **Overall Benefit:**
 - Achieves **Increased Relevance & Performance** by dramatically reducing "embedding noise" from irrelevant details and significantly speeding up the retrieval process.

Natdhanai Praneenatthavee, AI engineer
เจ้าของเพจ ตื่นมาโค้ด python

RETRIEVER

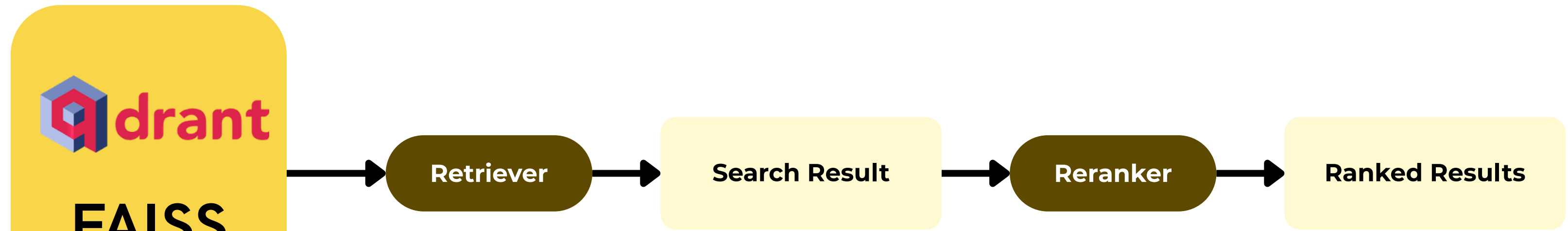
Retriever

Text search, Vector search, Hybrid search, SQL query

Criteria	Keyword Search	Vector Search	Hybrid Search
Search Method	Lexical, based on exact word matches	Semantic, based on meaning via embeddings	Combines lexical and semantic search
Libraries	ElasticSearch, Whoosh, BM25	FAISS, Chroma, PgVector	EnsembleRetriever (LangChain)
Accuracy	Good for exact queries	High contextual accuracy	Best overall (balanced)
Speed	Very fast	Slightly slower due to embedding computation	Moderate (depends on setup)
Use Case	Legal text, keyword-heavy data	Q&A, contextual retrieval	Knowledge base or enterprise search
Weakness	Fails with synonyms or paraphrasing	Can retrieve irrelevant but semantically close docs	Requires more computation and tuning
Example Scenario	Search for 'invoice' returns exact matches only	Search for 'bill' also retrieves 'invoice'	Combines both results for maximum recall

ENHANCE SEARCH WITH RERANKER

DATABASE



```
Query: "ไทยรุ่งยูเนี่ยนคาร์ อยู่ไหน"
Result:
{
  {
    document_id: "124",
    document_name: "รุ่นรถของ ไทยรุ่งยูเนี่ยนคาร์"
  },
  {
    document_id: "125",
    document_name: "ขั้นตอนผลิต ของไทยรุ่งยูเนี่ยนคาร์"
  },
  {
    document_id: "123",
    document_name: "ไทยรุ่งยูเนี่ยนคาร์ ที่ตั้ง"
  }
}
```

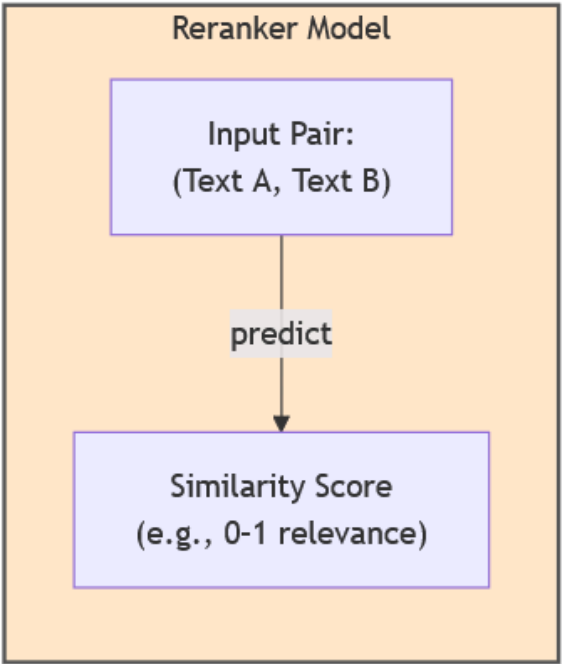
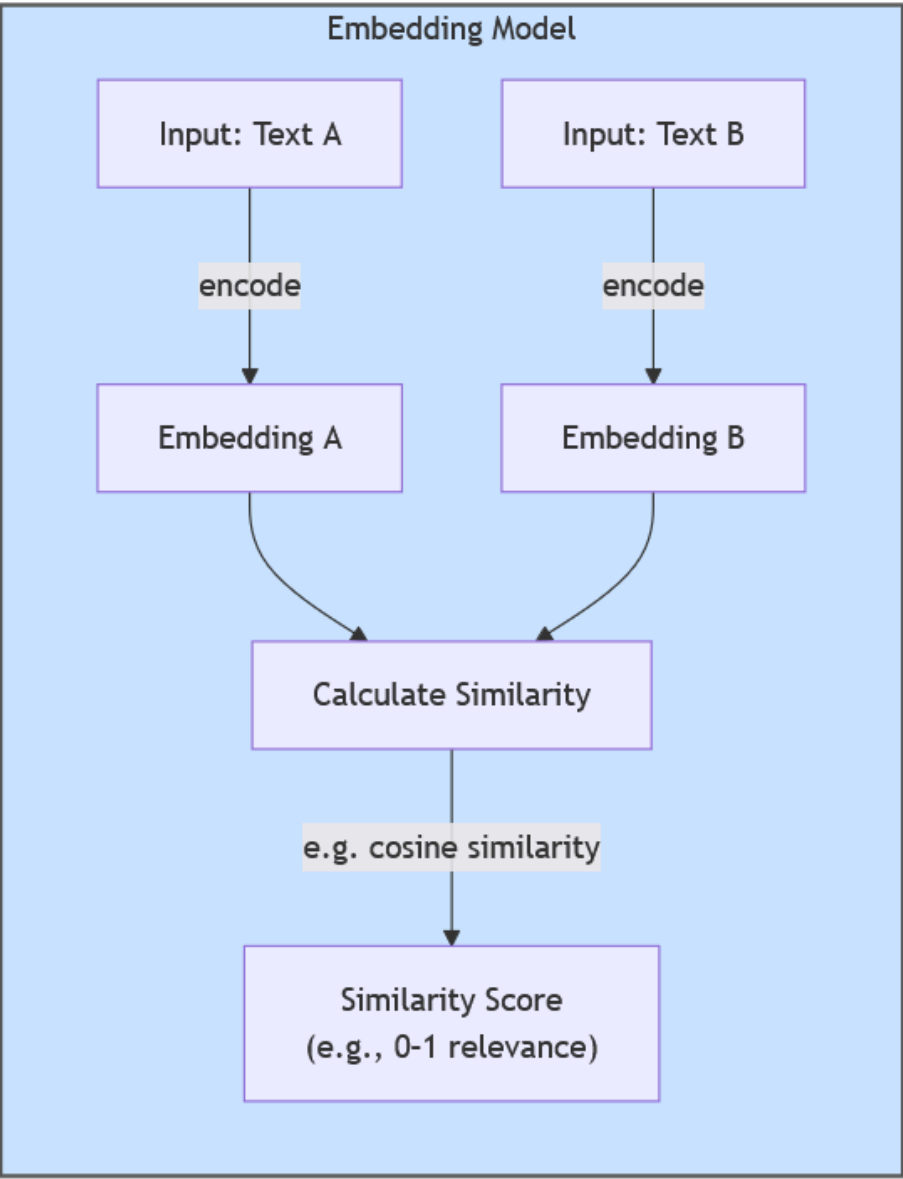
```
Ranked Results:
{
  {
    document_id: "123",
    document_name: "ไทยรุ่งยูเนี่ยนคาร์ ที่ตั้ง"
  },
  {
    document_id: "124",
    document_name: "รุ่นรถของ ไทยรุ่งยูเนี่ยนคาร์"
  },
  {
    document_id: "125",
    document_name: "ขั้นตอนผลิต ของไทยรุ่งยูเนี่ยนคาร์"
  }
}
```

RERANKER

Reranker

The answer is that rerankers are much more accurate than embedding models.

Type	Model Example	Description	Best For
Cross-Encoder	cross-encoder/ms-marco-MiniLM-L-6-v2	Jointly encodes query and document, highest accuracy	Precise matching, small datasets
Bi-Encoder (Embedding)	sentence-transformers/all-MiniLM-L6-v2	Separately embeds query and docs for speed	Large-scale search, lower cost
LLM-based Reranker	GPT-based custom reranker	Uses LLM reasoning for contextual ranking	Complex semantic tasks



LLM-BASED RESPONSE GENERATION

LLM (LARGE LANGUAGE MODEL)

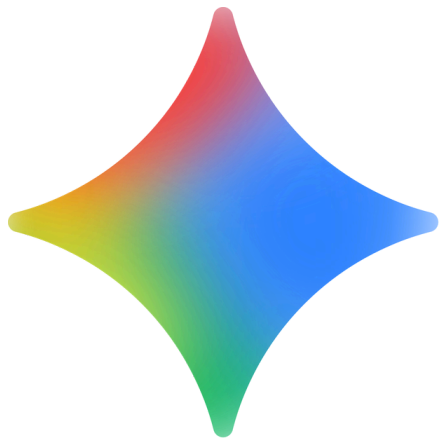
LLM

Use LLM to generate responses with retrieved context

Criteria	Open-Source LLMs	Closed-Source (Commercial) LLMs
Examples	Llama 3, Mistral, Falcon, Gemma	GPT-4o, Claude 3, Gemini, etc.
Deployment	Local / On-premise	Cloud API via provider
Customization	Full fine-tuning & retraining possible	Limited customization (prompt tuning only)
Performance	Varies with hardware and tuning	High, optimized by vendor
Data Privacy	High (self-hosted, no external API calls)	Depends on vendor compliance and data policies



deepseek

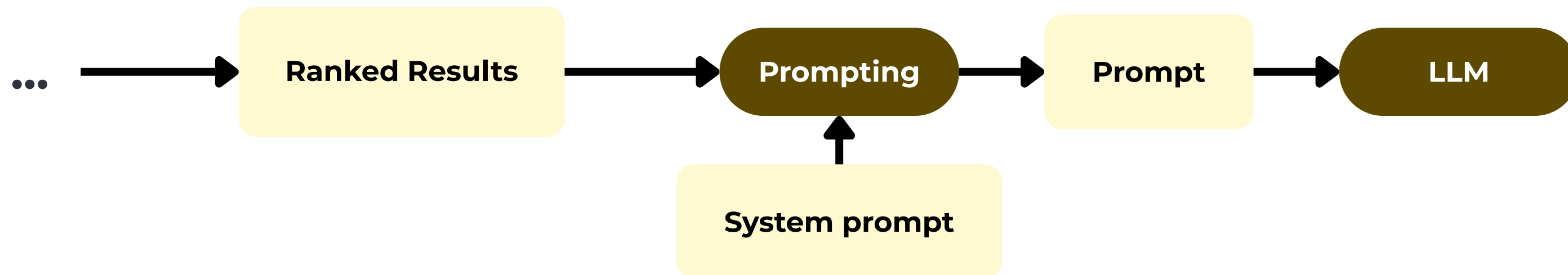


EXAMPLE: LLM-BASED RESPONSE GENERATION –

User Query: "ไทยรุ่งยูเนี่ยนคาร์ ทำธุรกิจอะไร?"

Retrieved Context (from www.thairung.co.th):

- ก่อตั้ง พ.ศ. 2510 โดย นายวิเชียร เผือกโอช
- ผู้ผลิต และ ประกอบ ตัวถัง รถยนต์ ไทย รายแรก ที่มี ครบวงจร ตั้งแต่ ออกแบบ ผลิต ไปจนถึง การขาย
- มีผลิตภัณฑ์ เช่น รถ SUV MPV และ รถ เฉพาะกิจ (Special Purpose Vehicles)
- ได้รับ มาตรฐาน ISO 9001, ISO/TS 16949 และ ISO 14001



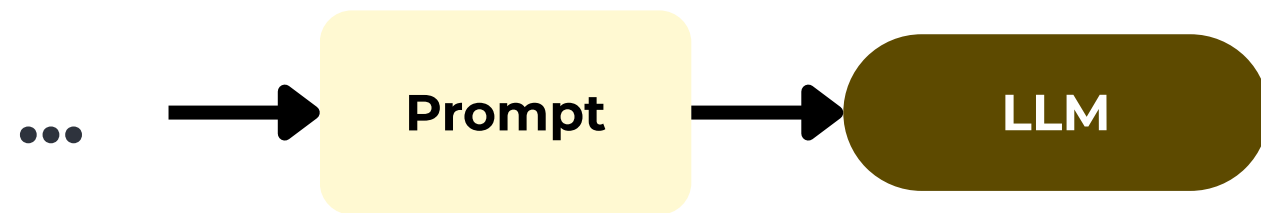
You are a knowledgeable assistant specializing in the Thai automotive industry. Answer based strictly on the provided context about Thai Rung Union Car (TRU). If the answer cannot be found in the context, say “ไม่พบข้อมูลในเอกสารที่ให้มา” (Information not found). Avoid adding opinions or external knowledge.

Context:
{context}

Question:
{question}

Answer in Thai with clear, factual sentences.

EXAMPLE: LLM-BASED RESPONSE GENERATION – ไทยรุ่งยูเนี่ยนคาร์



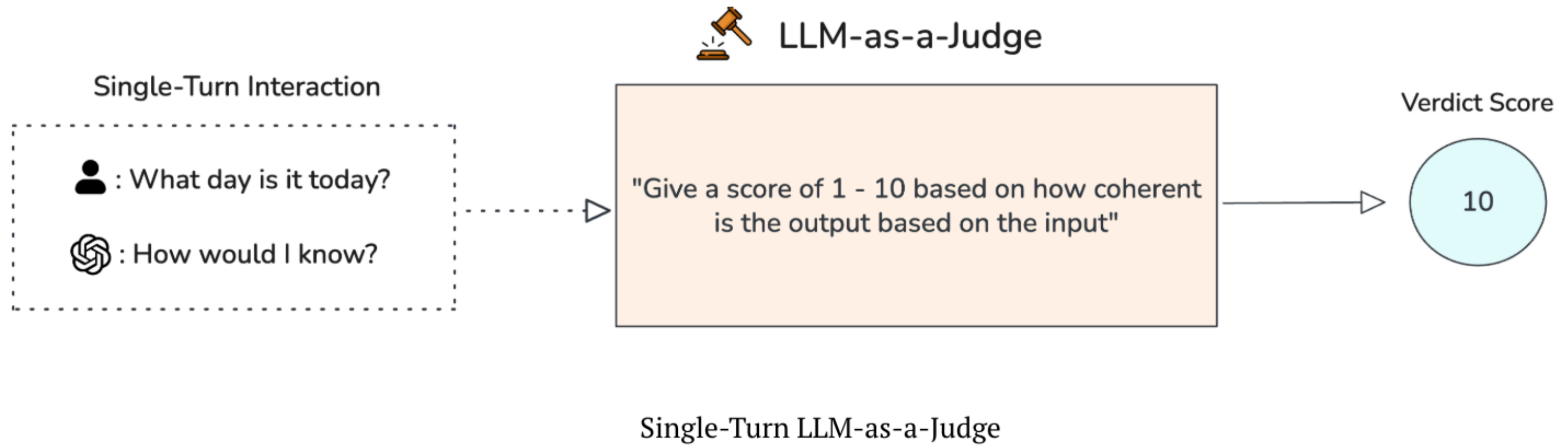
Answer:

ไทยรุ่งยูเนี่ยนคาร์ จำกัด (มหาชน) ดำเนินธุรกิจผลิตและประกอบตัวถังรถยนต์และยานยนต์เฉพาะกิจของไทยครบวงจร ตั้งแต่การออกแบบ ผลิต ไปจนถึงจำหน่ายทั้งในประเทศและต่างประเทศ

RAG EVALUATION

Aspect	What It Measures	Common Metrics
Retrieval	How relevant are retrieved documents?	Precision@k, Recall@k, nDCG, MRR
Generation	How accurate & fluent are the answers?	ROUGE, BLEU, BERTScore
Faithfulness	Are answers grounded in retrieved context?	LLM-as-a-Judge, Human Eval
End-to-End QA	Final answer correctness	F1, Exact Match

RAG EVALUATION



ADVANCED RAG TECHNIQUES

ADVANCED RAG TECHNIQUES

INTENT, NER, METADATA FILTERING

Intent Classification

Extract user intent:

“ขอที่อยู่ บริษัทหน่อย” → “FAQ”

“ขอราคารุ่น TR TRANSFORMER II SWB 5 ที่นั่ง ” → “Product detail”

NER extraction

Extract NER: (Named-entity recognition)

“ขอราคารุ่น TR TRANSFORMER II SWB 5 ที่นั่ง”

→ “ขอราคารุ่น **TR TRANSFORMER II SWB 5 ที่นั่ง** ”

Product name

METADATA

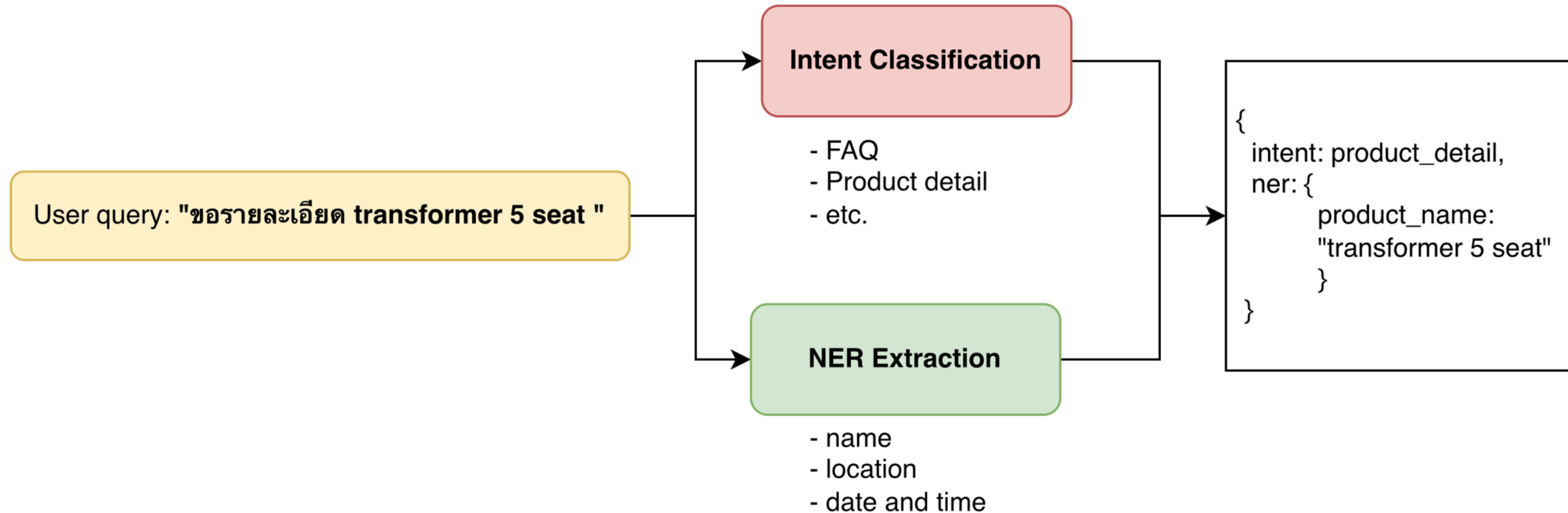
Adding Informative in User query

“ขอราคารุ่น TR TRANSFORMER II SWB 5 ที่นั่ง ”

→ intent: product_detail, ner: { product_name: "transformer 5 seat", metadata: ราคา, transformer, 5 ที่นั่ง

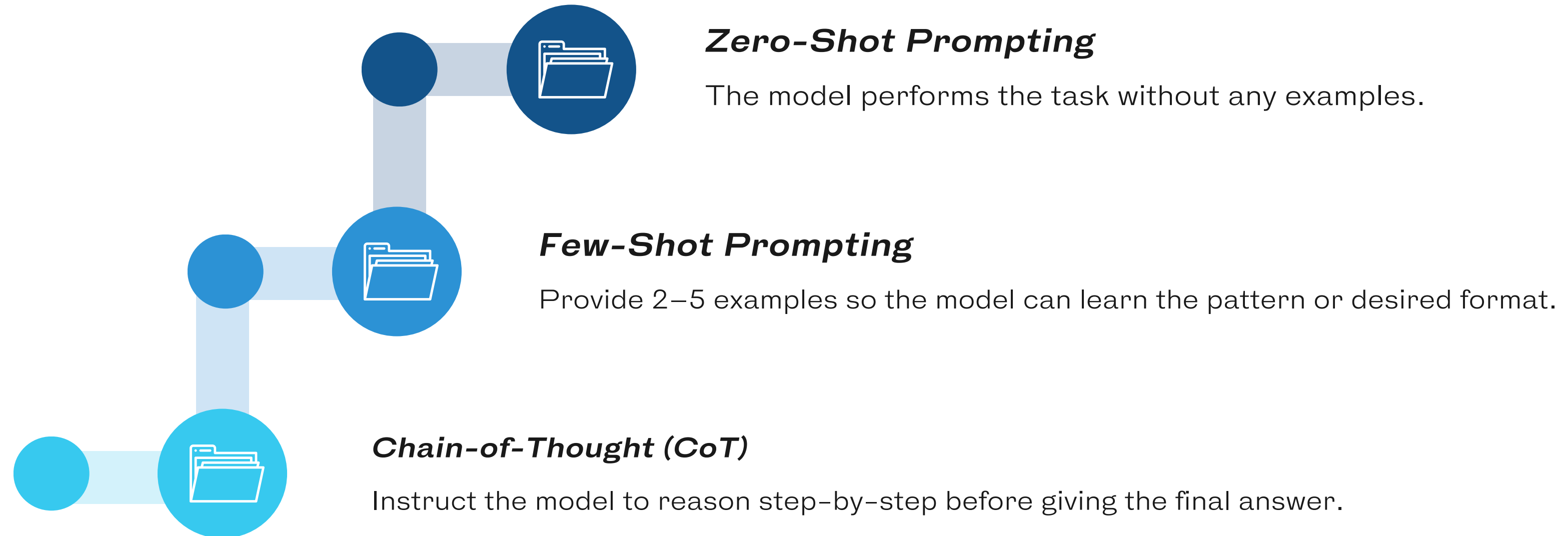
ADVANCED RAG TECHNIQUES

INTENT, NER, METADATA FILTERING



ADVANCED PROMPT ENGINEERING

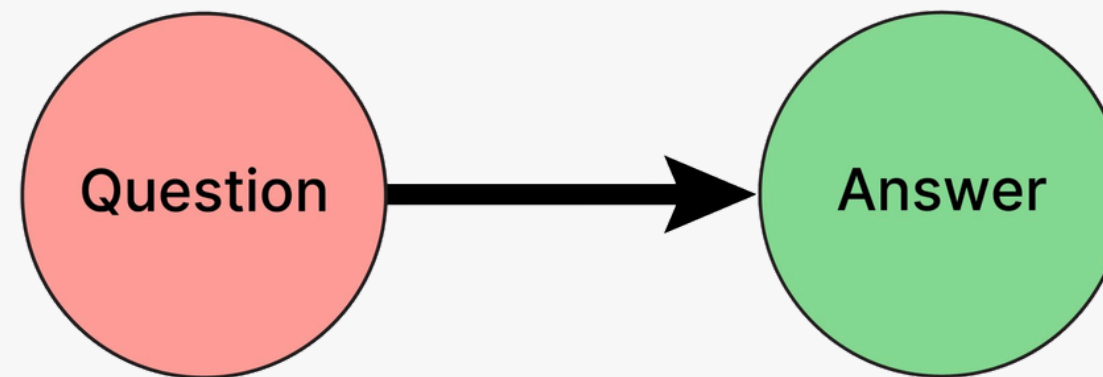
FOR RAG



ADVANCED PROMPT ENGINEERING

FOR RAG

Zero-Shot Prompting



Zero-shot prompting means giving the model an instruction **without any examples**. The model relies entirely on its pre-trained knowledge to generate an output.

Key Points

- Fast and efficient
- Useful for simple tasks or tasks where the context retrieved by RAG already provides enough information.
- Performance drops for complex reasoning tasks.

ADVANCED PROMPT ENGINEERING

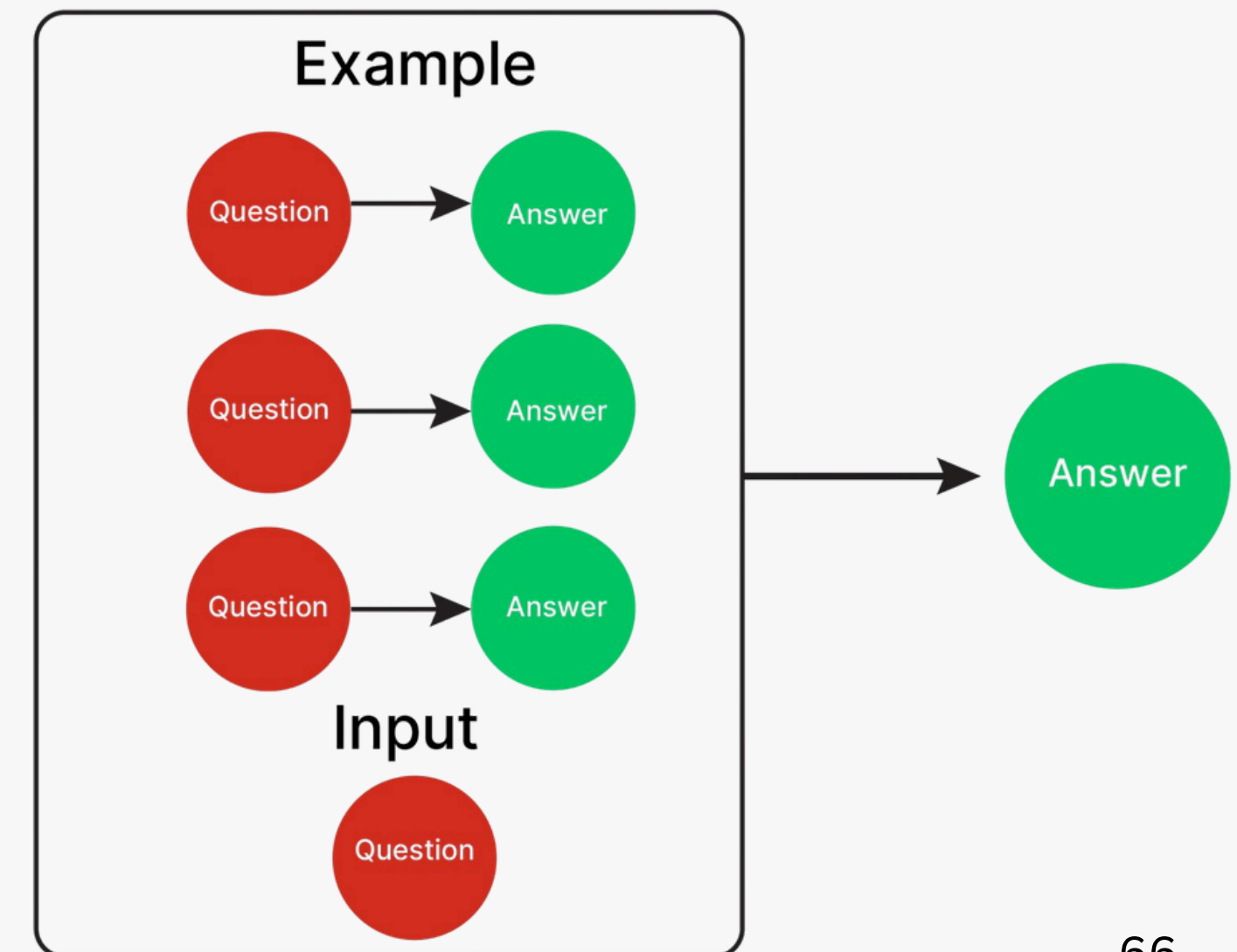
FOR RAG

Zero-Shot Prompting

Few-shot prompting provides a **few examples (input-output pairs)** in the prompt so the model can learn the desired pattern or structure.

Key Points

- Helps the model understand the expected output style, reasoning pattern, and structure.
- Useful when the RAG system retrieves content with structured patterns.
- long prompts, higher cost, and context-window issues.



ADVANCED PROMPT ENGINEERING

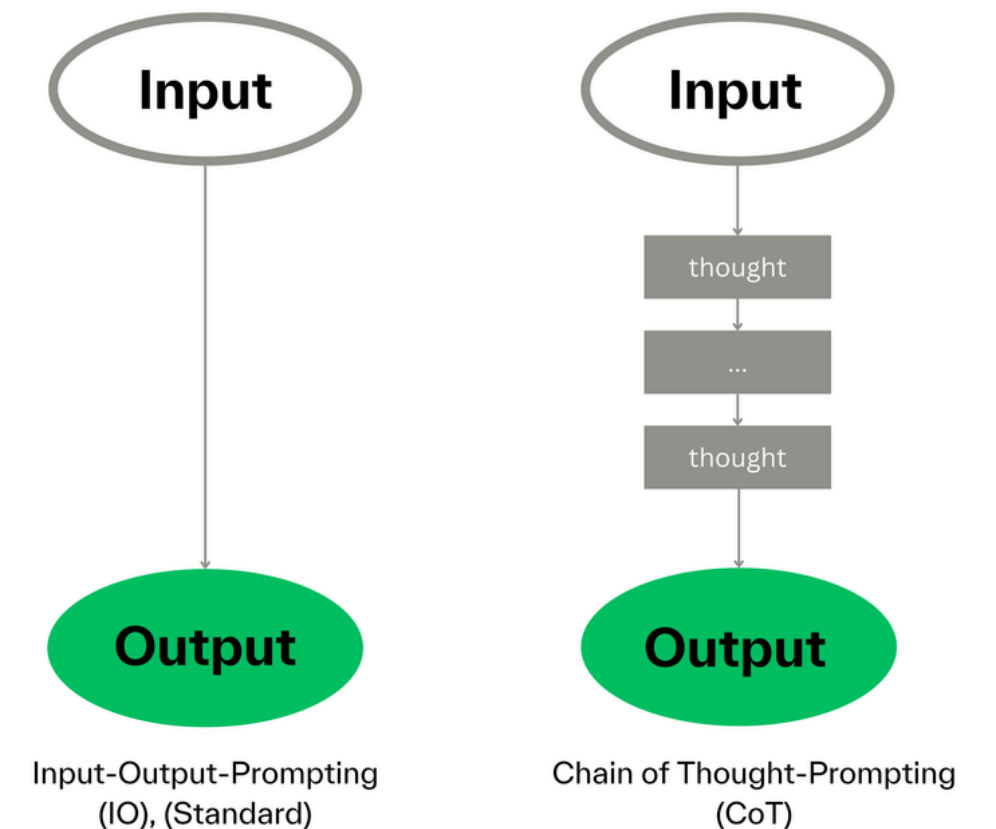
FOR RAG

Chain-of-Thought (CoT) Prompting

Prompting is a technique for writing prompts that **encourages AI models to show their reasoning process step-by-step** before arriving at a final answer, rather than jumping directly to the conclusion.

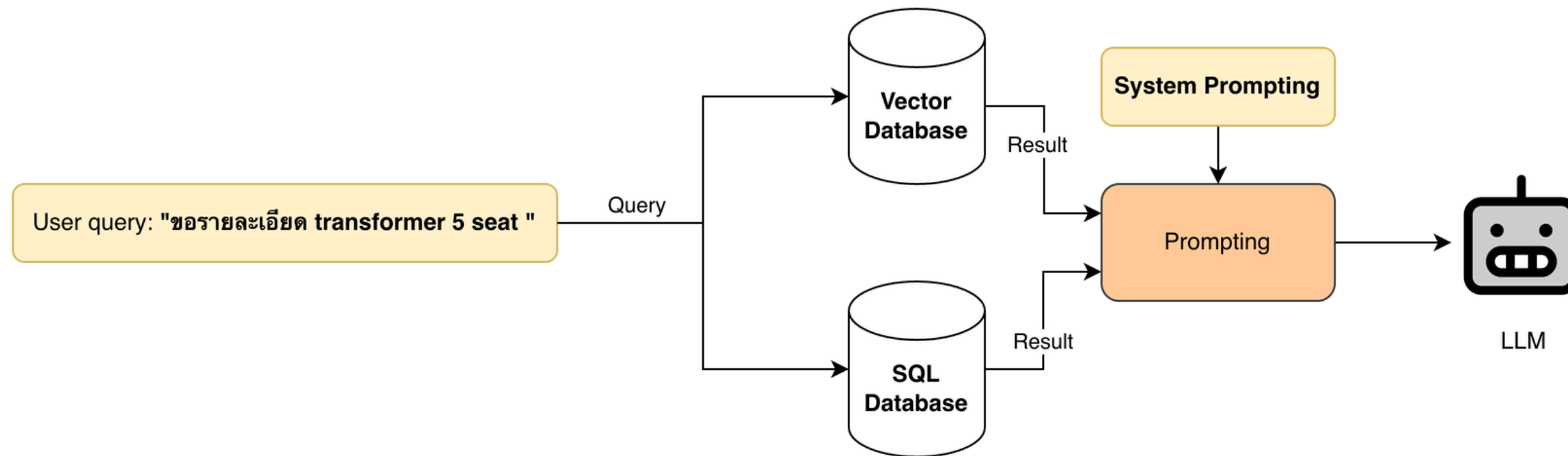
Key Points

- **Improves accuracy** especially for complex problems
- **Enables verification** of where the model's reasoning may have gone wrong
- **Provides transparency** into the logic behind answers



ADVANCED PROMPT ENGINEERING

FOR PARALLEL RETRIEVER FOR REDUCE LATENCY



Key Benefits:

- **Reduced Latency:** Queries run in parallel instead of sequentially
- **Faster Response Time:** No waiting for one database before querying another
- **Rich Context:** Combines both semantic (vector) and structured (SQL) data
- **Better Answers:** LLM gets comprehensive information from multiple sources

ADVANCED PROMPT ENGINEERING

FOR TOKEN-ORIENTED OBJECT NOTATION (TOON)

Token-Oriented Object Notation

TOON

Compact, human-readable serialization of JSON data for LLM prompts

Workflow:

JSON

→ encode()

→ TOON

→ LLM

∅ Tokens:

▶ Try it:

→ npx @toon-format/cli *.json

≈30-60% less

	avg tokens	retrieval accuracy	best for
TOON →		73.9%	repeated structure • tables varying fields • deep trees
JSON →		69.7%	

JSON

```
{
  "reviews": [
    {
      "id": 101,
      "customer": "Alex Rivera",
      "rating": 5,
      "comment": "Excellent!",
      "verified": true
    },
    {
      "id": 102,
      "customer": "Brij Pandey",
      "rating": 5,
      "comment": "Game changer!",
      "verified": true
    },
    {
      "id": 103,
      "customer": "Casey Lee",
      "rating": 3,
      "comment": "Average",
      "verified": false
    }
  ]
}
```

JSON SIZE

412 chars

TOON

```
reviews[3]{
  id, customer, rating,
  comment, verified}:

101, Alex Rivera, 5,
Excellent!, true

102, Brij Pandey, 5,
Game changer!, true

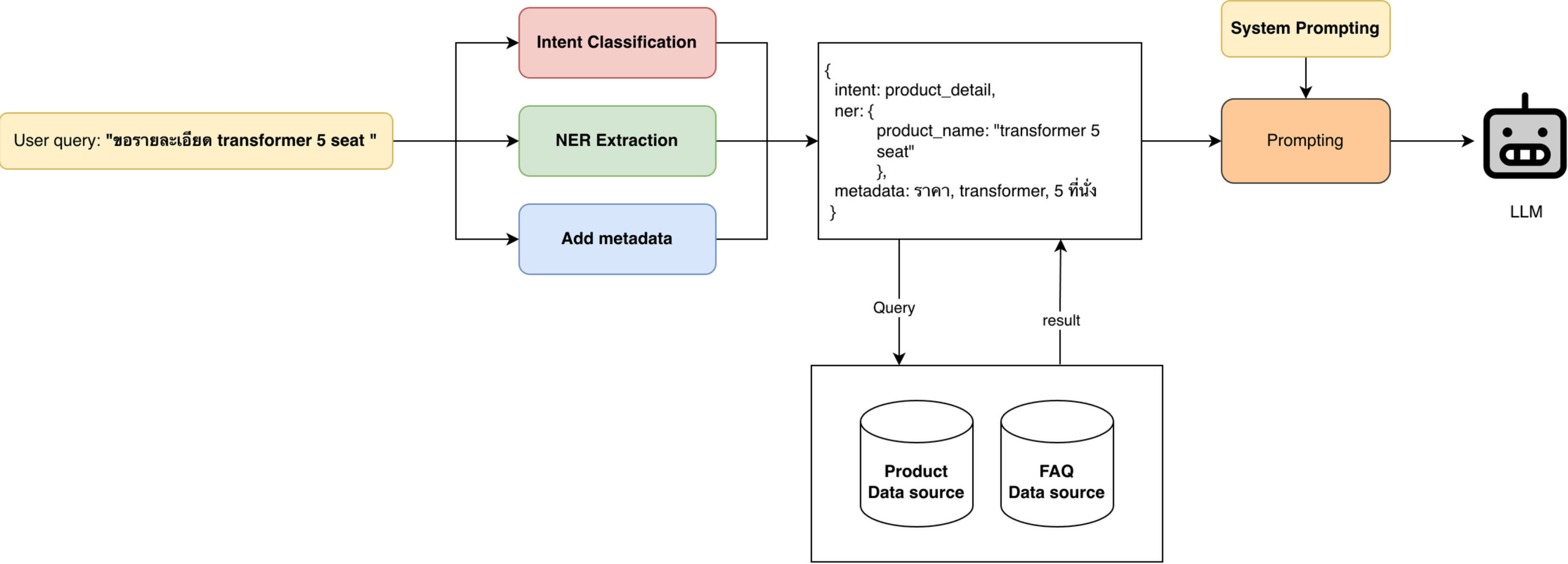
103, Casey Lee, 3,
Average, false
```

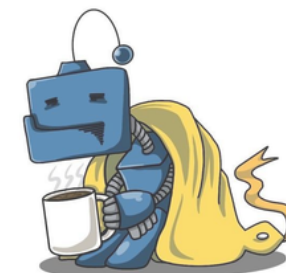
TOON SIZE

154 chars

RAG ARCHITECTURE DESIGN

EXAMPLE DESIGN





MLFLOW

AI LIFECYCLE MANAGEMENT

Natdhanai Praneenatthavee, AI engineer
Founder ทีมมาโค้ด python

MLFLOW



Tracking

Record and query experiments: code, data, config, results

Projects

Packaging format for reproducible runs on any platform

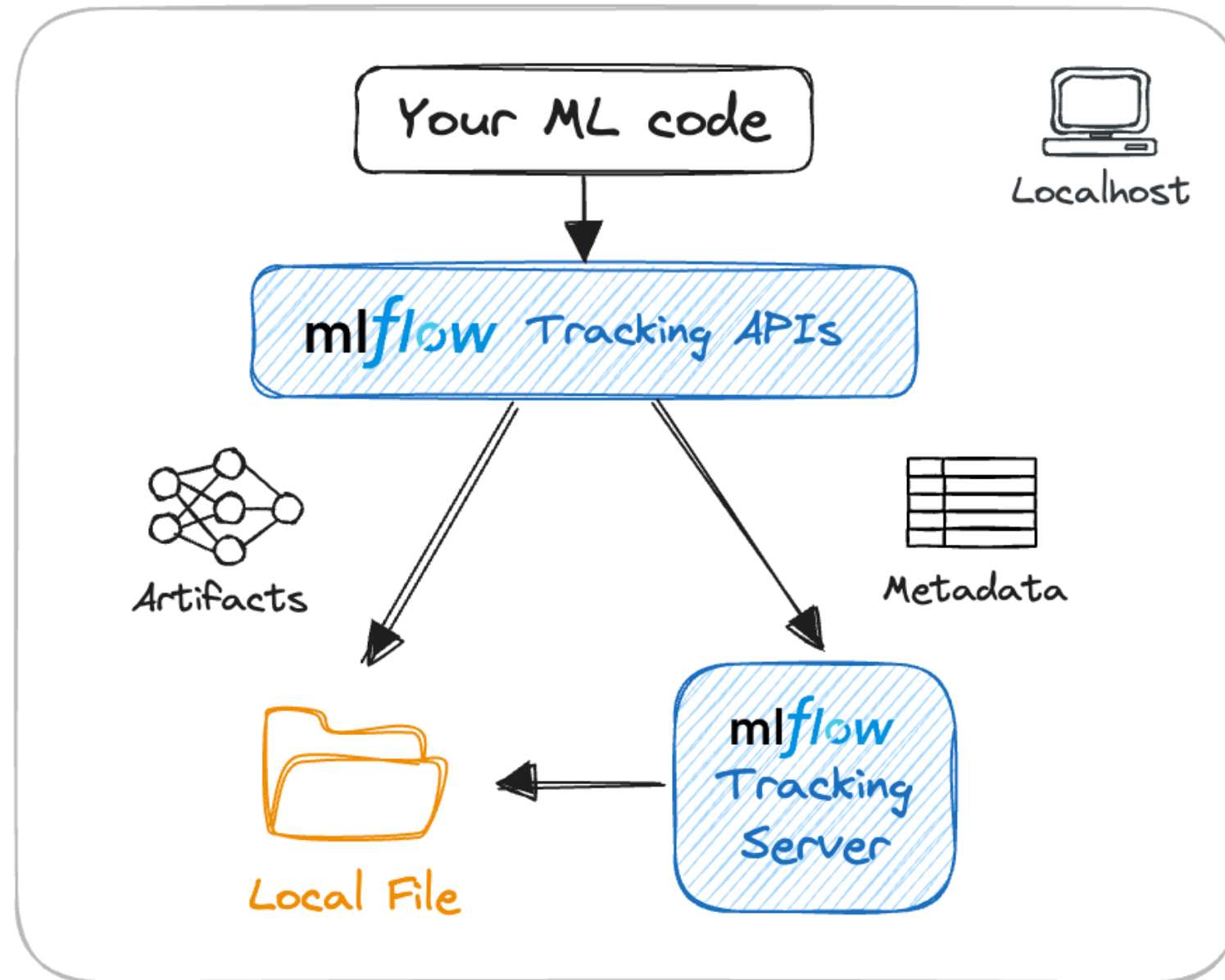
Models

General format for sending models to diverse deploy tools

MLflow คือแพลตฟอร์ม Open Source ที่ใช้สำหรับบริหารจัดการวงจรชีวิตของ Machine Learning (ML Lifecycle) ตั้งแต่เริ่มทดลอง ไปจนถึงนำไปใช้งานจริง

เปรียบเสมือน 'สมุดบันทึกการทดลองอัจฉริยะ' ของ DataSci ที่ช่วยจดจำว่าเราปรับค่าอะไรไปบ้าง ผลลัพธ์เป็นอย่างไร และช่วยแพ็ค Projects ส่งมอบให้ทีมนำไปใช้ต่อได้อย่างเป็นระเบียบ

MLFLOW



4 เสาหลักของ MLflow

1. Experiment Tracking (การติดตามการทดลอง):

- บันทึกค่าพารามิเตอร์ (Parameters), โค้ด, และผลลัพธ์ (Metrics) ของการรันแต่ละครั้ง
- ประโยชน์: ทำให้เรารู้ว่า "โมเดลเวอร์ชันไหนเก่งที่สุด" และ "ทำซ้ำได้ (Reproducible)"

2. Models (การจัดการโมเดล):

- ห่อหุ้ม (Package) โมเดลให้อยู่ในรูปแบบมาตรฐาน (Standard format)
- ประโยชน์: ไม่ว่าจะเขียนด้วย Library อะไร (Scikit-learn, TensorFlow, PyTorch) ก็สามารถนำไปรันต่อได้ทุกที่

3. Model Registry (ทะเบียนคุมโมเดล):

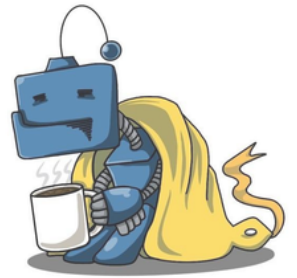
- ระบบจัดการเวอร์ชัน (Versioning) และสถานะของโมเดล (Staging vs Production)
- ประโยชน์: ควบคุมได้ว่าโมเดลตัวไหนพร้อมใช้งานจริง ป้องกันความสับสน

4. Evaluation (การประเมินผล): สำคัญมากสำหรับ LLM/RAG

- เครื่องมือวัดประสิทธิภาพและความแม่นยำของโมเดล

CONTEXT ENGINEERING

EFFECTIVE CONTEXT ENGINEERING FOR AI AGENTS



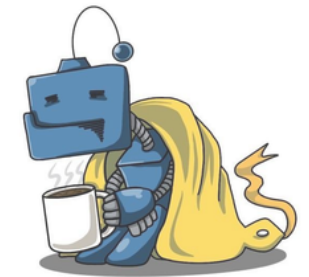
Context is a critical but finite resource for AI agents. In this post, we explore strategies for effectively curating and managing the context that powers them.

"อย่าเน้นที่ปริมาณ แต่ให้เน้นที่คุณภาพของข้อมูล" การทำ AI Agent ให้เก่ง ไม่ใช่การให้มันอ่านทุกอย่าง แต่คือการรู้จักเลือกส่งข้อมูลที่สำคัญที่สุดให้มันในเวลาที่เหมาะสม

1. **"Context is a critical resource"** (บริบทคือทรัพยากรที่สำคัญ)

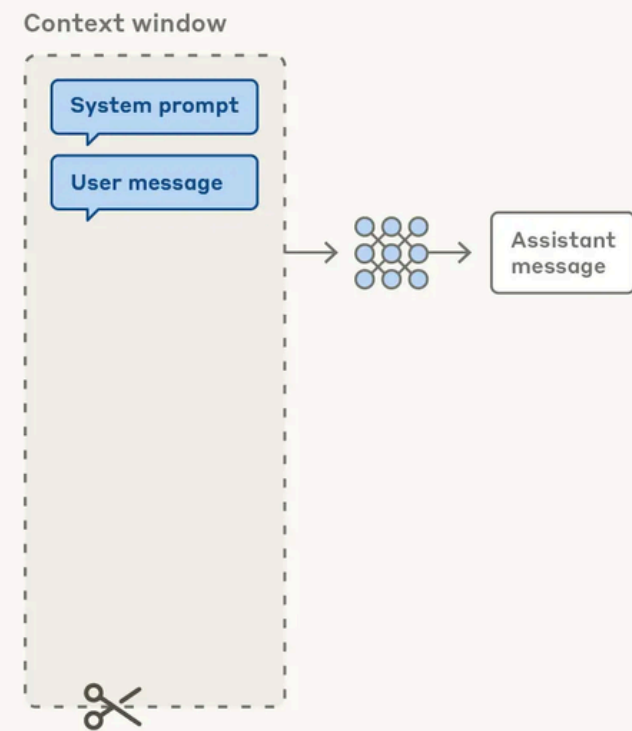
- **หมายถึง:** ข้อมูลที่คุณใส่เข้าไปใน Prompt (เช่น คำสั่ง, ตัวอย่าง, ไฟล์เอกสาร, ประวัติการคุย) คือสิ่งที่กำหนดว่า AI จะฉลาดหรือทำงานได้ตรงใจแค่ไหน
- **เปรียบเทียบ:** เหมือน "วัตถุดิบ" ในการปรุงอาหาร ถ้าวัตถุดิบดีและครบถ้วน อาหารก็ออกมาดี

CONTEXT ENGINEERING VS. PROMPT ENGINEERING

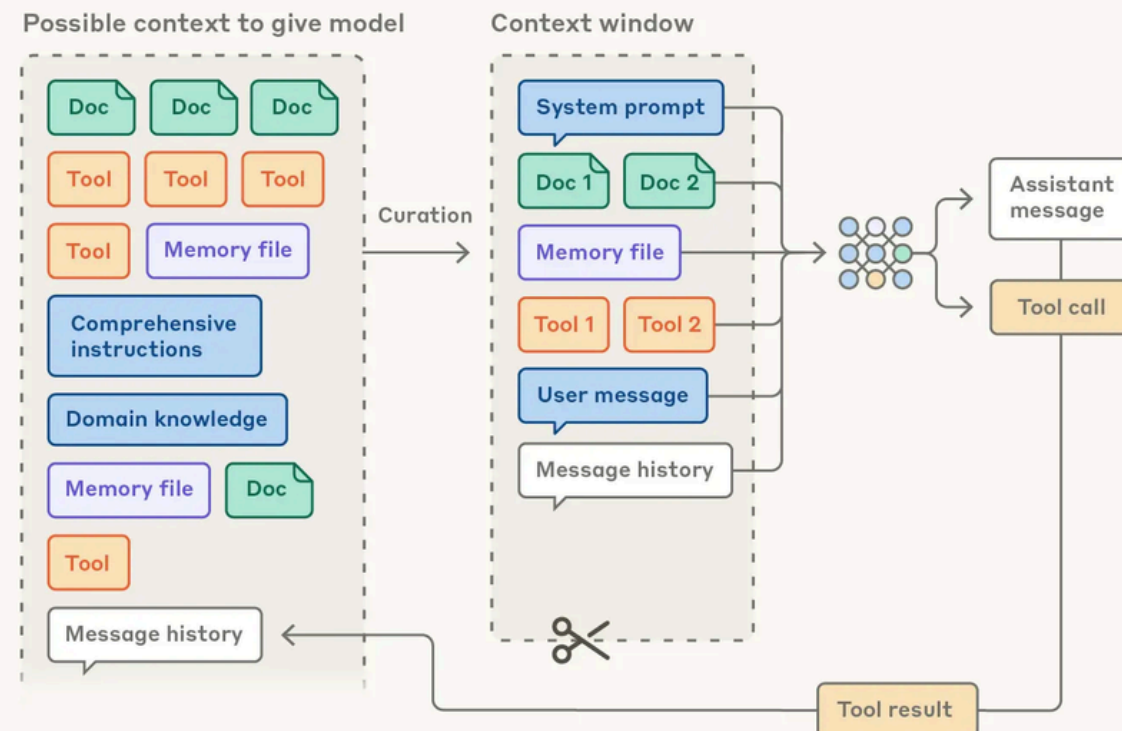


Prompt engineering vs. context engineering

Prompt engineering for single turn queries



Context engineering for agents



Prompt Engineering (วิธีเก่า - สำหรับงานแบบ one-shot)

ใช้กับ:

- การสนทนาแบบครั้งเดียวจบ (single-turn)
- งานแบบ classification หรือ text generation ที่ทำครั้งเดียว
- ตัวอย่าง: "จำแนกอีเมลนี้ว่าเป็น spam หรือไม่"

มุ่งเน้นที่:

- เขียน prompt ให้ดี โดยเฉพาะ system prompt
- ออกแบบคำสั่งให้ชัดเจน มีตัวอย่าง มีโครงสร้างที่ดี
- Context ส่วนใหญ่คือ: System prompt + User message เท่านั้น

Context Engineering (วิธีใหม่ - สำหรับ AI Agents)

ใช้กับ:

- AI Agents ที่ทำงานหลายรอบ (multi-turn)
- งานที่ซับซ้อน ใช้เวลานาน ต้องใช้หลาย tools
- ตัวอย่าง: ระบบช่วยแก้ปัญหา gas turbine ที่ต้องค้นหาเอกสาร ถึงข้อมูล วิเคราะห์ไปเรื่อยๆ

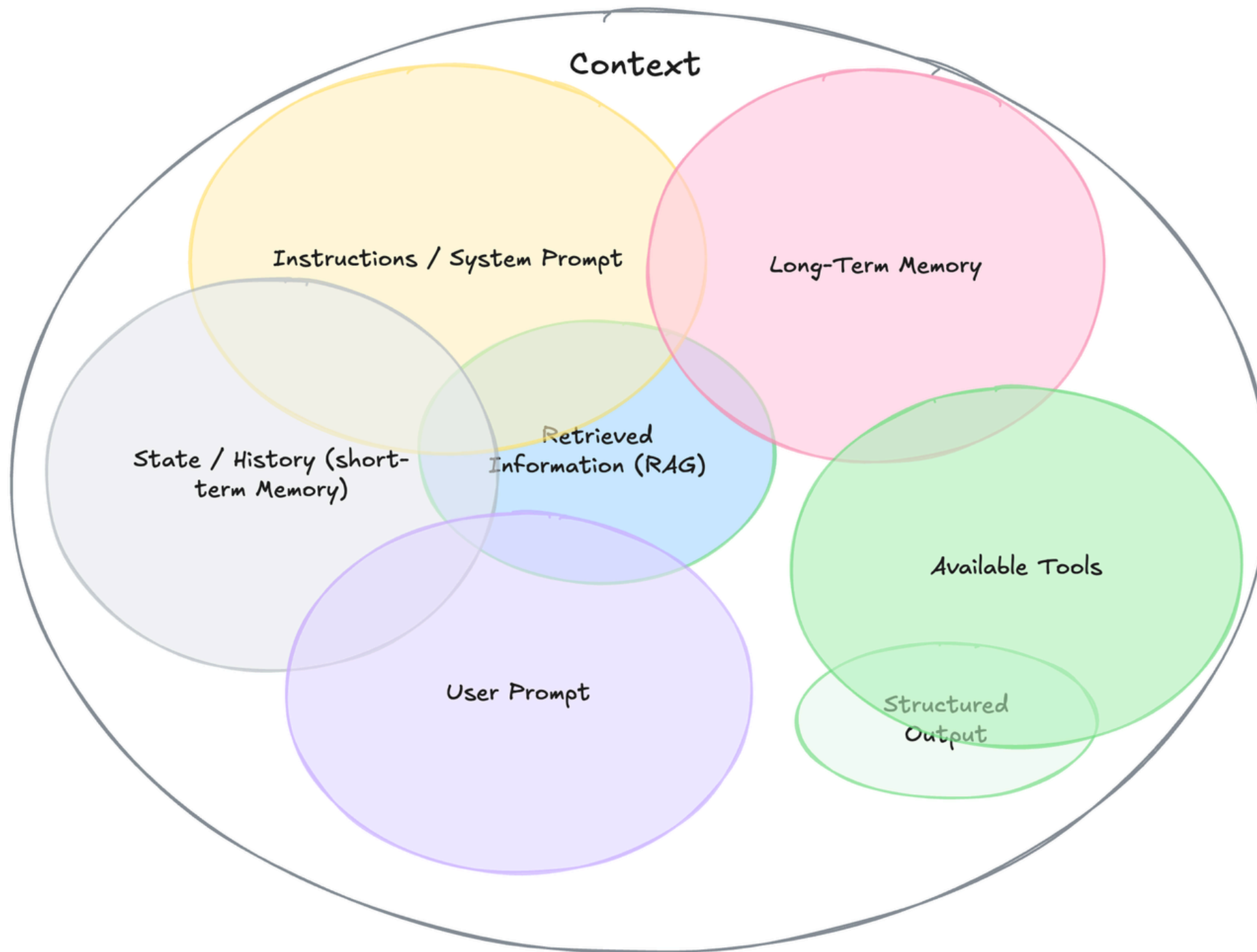
มุ่งเน้นที่:

- จัดการ context ทั้งหมด ไม่ใช่แค่ prompt
- เลือกว่าจะใส่ข้อมูลอะไรเข้า context window

PROMPT → CONTEXT ENGINEERING?



Context Engineering



ทำไมต้องเปลี่ยนจาก Prompt → Context Engineering?

ปัญหาของ AI Agents:

1. ข้อมูลเพิ่มขึ้นเรื่อยๆ (constantly evolving universe)

- ทุกครั้งที่ Agent ทำงาน จะมีข้อมูลใหม่เกิดขึ้น
- Tool results, การค้นหา, ข้อมูลจาก MCP servers
- Message history ที่ยาวขึ้นเรื่อยๆ

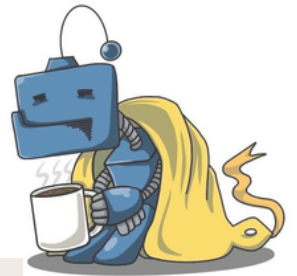
2. Context Window จำกัด (limited context window)

- ไม่สามารถใส่ทุกอย่างได้
- ต้องเลือกว่าจะอะไรสำคัญสำหรับรอบถัดไป

3. ต้องปรับ (refine) แบบวนลูป (cyclically refined)

- ทุกรอบต้องประเมินใหม่
- เอาข้อมูลเก่าที่ไม่จำเป็นออก
- เพิ่มข้อมูลใหม่ที่เกี่ยวข้องเข้าไป

THE ANATOMY OF EFFECTIVE CONTEXT

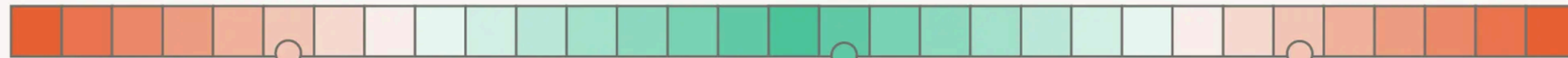


Calibrating the system prompt

Too specific

Just right

Too vague



You are a helpful assistant for Claude's Bakery.
You must respond to the name Claude.
For every user request you MUST FOLLOW THESE STEPS:

1. Identify the user intent as one of the following: ["incident_resolution", "general_inquiry", "order_resubmission", "account_maintenance", "requires_escalation"]
2.
 - if user intent is "incident_resolution", ask 3 followup questions to gather information, then always call the resolve tool
 - if user intent is "general_inquiry", do not ask followup questions and answer in one shot
 - if user intent ...
 - ...
3. Here is an exhaustive list of cases that should be tagged as "requires_escalation":
 - If the intent is incident_resolution but the user is in a different country
 - If the user left a physical belonging in the store
 - ...
4. Once you've ruled out escalation scenarios you should consider all the tools at your disposal.
5. If the user_request contains an order_id you should tag the user intent as "order_resubmission", unless the user meets 5/7 of the following requirements:
 - User is asking for time update
 - User is asking for location update
 - ...
6. If the user wants to request a new order, but they already have another order in flight, you should follow these 5 steps of the resolution procedure:
 - (1) Call check_order tool to see where the current order is
 - ...

...

You are a customer support agent for Claude's Bakery.
You specialize in assisting customers with their orders and basic questions about the bakery. Use the tools available to you to resolve the issue efficiently and professionally.

You have access to order management systems, product catalogs, and store policies. Your goal is to resolve issues quickly when possible. Start by understanding the complete situation before proposing solutions, ask follow-up questions if you do not understand.

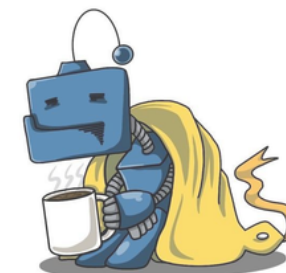
Response Framework:

1. Identify the core issue - Look beyond surface complaints to understand what the customer actually needs
2. Gather necessary context - Use available tools to verify order details, check inventory, or review policies before responding
3. Provide clear resolution - Offer concrete next steps with realistic timelines
4. Confirm satisfaction - Ensure the customer understands the resolution and knows how to follow up if needed

Guidelines:

- When multiple solutions exist, choose the simplest one that fully addresses the issue
- If a user mentions an order, check its status before suggesting next steps
- When uncertain, call the human_assistance tool
- For legal issues, health/allergy emergencies, or situations requiring financial adjustments beyond standard policies, call the human_assistance tool
- Acknowledge frustration or urgency in the user's tone and respond with appropriate empathy

You are a bakery assistant, you should attempt to solve customers issues in a manner consistent with the principles and essence of the company brand. Escalate to a human if needed.



THANK YOU

Natdhanai Praneenatthavee, AI engineer
Founder ทีมมาโค้ด python